

EM Stories pivot

Things from interviews I need to ensure I plan for

- How did you grow a member of team
- how did you help your team clear out an issue
-

Opening

- Write down the exact words you want to say. rehearse it till you nail the delivery, nail the first minute.
- Confidence is something you put on.
- Watch / listen to your recordings
- You have the floor, own it. → don't shrink, own it.

Austen McDonald defines eight signals in his book which he describes as the broad categories that almost all behavioral interview questions are designed to assess.

- **Driving Results(Impact)**: Demonstrate that you can deliver business value and measurable outcomes with your work
- **Handling Ambiguity**: Showing how you operate when requirements are unclear, priorities shift, or you have to make progress with incomplete information
- **Conflict Resolution**: Your ability to navigate disagreements with peers, managers, or cross-functional partners productively.
- **Growing Continuously**. Evidence that you seek feedback, learn from mistakes, and are actively improving your skills.
- **Communicating Effectively**: How you convey complex technical ideas, manage stakeholders, and tailor your message to different audiences.
- **Ownership**: Taking full responsibility for a project or problem space from start to finish, including things outside your narrow job description.

- **Leadership & Influence** : Your ability to align people around a technical vision, drive cross-team initiatives, and lead without formal authority.
- **Mentorship**: How you invest in others, help more junior folks grow, and improve the overall talent density of your team or organization.

"Hi, my name is [Name]. I'm a Senior Engineering Manager at [Bank ABC], where I lead global teams focused on **data platforms and large-scale application modernization in the cloud**. Our systems process **high-volume event data across multiple regions**, supporting critical business workflows.

Before that, I was at **Mastercard**, leading the **blockchain engineering team** that migrated Mastercard's blockchain infrastructure to **Layer 2 rollups**, improving scalability and transaction efficiency.

Across both roles, I've focused on building **high-performing, globally distributed teams** that deliver **reliable, scalable, and business-critical platforms**."

hi my name is Sogo. I am a senior EM at bank Citi, where I lead a global team focussed on application modernization/cloud migration and data platforms. We process high volume events daily across multiple regions. Before that, I was at Mastercard working on the blockchain engineering team, where I lead the team that migrated mastercard blockchain to work on L2 rollups off chain for increased scale. I like to focus on building high performing teams that build platforms that are reliable, scalable and business critical

Why became a manager

My path to management was driven by my interest in the *outcome* of the work. While I'm technically grounded, I'm highly motivated by the final product and its success.

I realized the most effective way to influence that outcome was by acting as a 'force multiplier' for the team. I find it deeply fulfilling to help a group of talented people collaborate effectively, remove obstacles, and achieve ambitious goals together. This started with a Tech Lead Manager role at Mastercard, and as our teams

delivered, my leadership responsibilities grew, solidifying that this was the right path for me.

- Reorganize your team based on an initiative
- Managed an underperformer
-

My current throughput reality check -

- 2B/day = 23,148 events/sec baselin.
 - Data volume (rough math)
 - 200 B event → 4.6 MB/s
 - 500 B/event → 11.6 MB/s
 - 1 KB/event → 23.1 MB/s
 -

"We process ~2B events/day from ~200 sources. We're standardizing on **Flink as a unified engine** for streaming and batch: real-time contracts/quality/enrichment from Kafka, and bounded backfills from HDFS/S3 into **Hudi/Iceberg**. We keep Hadoop storage (HDFS/Hive) and only use Spark where it's uniquely valuable. This cut pipeline sprawl, gave us **exactly-once end-to-end**, and aligned real-time and batch SLAs under one runtime."

"The direction we're steering DRIFT is toward a real-time data fabric — Kafka feeding Flink for continuous transformation and quality enforcement, and Iceberg giving us ACID-compliant, versioned tables on HDFS/S3.

It's early — we're still in pilot phases with a few domains — but the architecture is already demonstrating that we can unify batch and stream workloads and bring policy enforcement closer to the data itself.

I'm not claiming we've perfected it, but I'm excited because it aligns exactly with what eBay is doing around governed, real-time data platforms."

Ambiguity- dependencies graph visualisation work story.

Team Setup

3 Teams

Reporting structure

Function	Data Ingestion	Data Services	Platform Team
Mission	Build/ manage pipelines from sources	Make data useful and accessible	Accelerate other teams with a stable secure, automated platform
Responsibilities	Messaging (kafka)., stream processing(flink), Data Validation	API's Microservices, data Modelling, warehousing, GQL	Core cloud infra, CI/CD. K8s, Monitoring and observability (Prometheus, Grafana, developer tooling)
Architect (2)	Infra architect guides pipeline scalability	Data architect shapes schema and serving strategy	both architects align platform standards
Manager	1	1	1
Tech Lead	1	1	1
Engineers	5	7	5
QA	1	1	1

- Clarity and focus - Each team has a clear, unambiguous mission, which reduce overlap and improves execution speed.
- High Autonomy - teams are organized around value streams (ingestion, Services)

- Scalability - This model is built to scale. As organization grows we can easily grow organically

Senior EM Story Pivot Table

Story	Leadership (People)	Execution (Delivery)	Technical Judgement	Impact (Outcomes)
1. Incident Empathy (Kafka OOM, 3am)	- Stayed calm under pressure and provided direction to first responders.- Built empathy for ICs by experiencing stress firsthand.- Fostered psychological safety during blameless post-mortem.	- Orchestrated incident response with clear task division.- Restored service within 45 minutes by sequencing broker restarts.- Documented and automated fix afterwards.	- Applied deep ops knowledge: sequential broker restart, monitoring ISR, consumer lag.- Recognized limits of self-managed Kafka → decision to move to managed service (MSK).	- Faster incident resolution.- Reduced 3am wakeups via automation.- Improved resilience by shifting to managed Kafka.- Stronger team culture around blameless learning.
2. Balancing Abstraction vs. Flexibility (Terraform + Policy-as-Code)	- Won developer trust by making the safe path the easiest path.- Handled pushback by allowing RFC-based exceptions.- Communicated empathetically: "guardrails, not gates."	- Delivered Terraform module registry + CLI scaffolding.- Integrated OPA/Sentinel guardrails into CI/CD.- Scaled adoption via office hours, examples, roadshows.	- Chose golden path coverage (REST, batch, event, data pipelines).- Defined strict vs. flexible knobs.- Prevented template rot with versioning + automated bumps.	- 85% adoption in 3 months.- IAM/network misconfig incidents ↓60%.- Time-to-first-deploy: weeks → hours.- Near-zero audit findings for cloud controls.
3. Top-Down & Bottom-Up	- Partnered with executives for	- Drove migration of	- Set KPIs for migration: %	- Zero high-severity audit

Story	Leadership (People)	Execution (Delivery)	Technical Judgement	Impact (Outcomes)
Alignment (Cloud Migration)	sponsorship (CISO, directors).- Engaged engineers via feedback sessions, demos, roadshows.- Used scorecards to create shared accountability.	70% of targeted workloads in 12 months.- Built developer portal for self-service provisioning.- Balanced migration pace with compliance gates.	workloads migrated, MTTR, SLA adherence.- Chose self-service + scorecards vs. ticket-based migration.- Designed compliance-in-the-pipeline.	findings.- Execs got visibility dashboards.- Developers got faster path to deploy.- Culture shifted from resistance → adoption.
4. SBOM Vulnerability Management (Your Own Story)	- Bridged security vs. engineering by clarifying ownership.- Rebuilt trust by removing noisy/unactionable tickets.- Positioned platform team as enabler, not blocker.	- Integrated SBOM tooling (Snyk, BlackDuck) into pipelines.- Built MongoDB-backed dependency graph.- Automated "cascading fix" via base image updates.	- Chose flexible graph DB to model dependencies.- Designed attribution logic to pinpoint true owners.- Established rules: platform patches infra, teams patch app-level.	- 90% reduction in misdirected tickets.- MTTR for inherited vulns dropped dramatically.- Saved hundreds of engineering hours.- Elevated platform team reputation.
5. Scaling Teams & Managing Managers	Coached new EMs; defined swim-lanes; setup 1:1s + manager circles; delegated roadmap ownership	Maintained delivery during 2X org growth; Scaled processes; enabled succession planning	Clear division of infra vs tooling teams; mentored managers in decision-making + prioritization	Two new managers grew into strong leaders (one promoted); org scaled 2x without burnout; high engagement + low attrition

Story	Leadership (People)	Execution (Delivery)	Technical Judgement	Impact (Outcomes)
6. Vision: Future of Cloud Ops - (FinOps + Platform Maturity)	Evangelized 3-year vision; aligned execs + engineers around shared cloud health language	Rolled out cloud health scorecards; introduced Finops dashboards; built resilience by default into templates	Integrated cost + compliance into developer portal; partnered with finance for cost attribution + recommendations	Cut Infra spend by 15%; execs tied funding to health metrics; engineers used dashboards for tuning; shared accountability created.
7. High-Performing Teams(Mini - Story A)	Defined clarity, autonomy, purpose; mentored managers; empowered IC growth	OKR-driven goals; doubled delivery capacity; created strong rituals	Used org design (infra vs tooling split) to reduce trash	Engagement ↑, attrition ↓; two managers promoted; culture = "autonomous but aligned."
8. Communication & Planning Structures	Created Transparency between product + eng; eliminated blind spots	Quarterly OKRs; bi-weekly roadmap reviews; jira dashboards; demo days	Structured planning + dependency management tools	Feature cycle time ↓20%; fewer surprises; stronger trust with product partners.
9. Inclusive Leadership & Diversity	Fostered inclusion: rotating leads; async design input, diverse hiring panels	embedded inclusivity into daily rituals mentorship programs	Process design for hiring fairness; async design practices for global teams.	+25% survey response "I felt heard" diverse hiring improved; higher retention; quieter voices drove key features

▼ Incident Empathy (Kafka OOM, 3am)

Situation

A few years ago, my team was running a critical part of our messaging infrastructure that dozens of downstream services depended on. One night, around 3 AM, I was paged because the entire cluster had gone into an out-of-memory state, which meant core applications across the business were effectively down. It was a customer-impacting incident with potential regulatory and financial implications if not resolved quickly.

Task

As the incident lead on call, my responsibility was twofold:

1. Restore service as quickly as possible to minimize business impact.
2. Coordinate communication so that senior stakeholders understood what was happening while engineers focused on recovery.

Action

- I immediately convened the on-call engineers in a bridge call and split responsibilities — one engineer on root cause investigation, another on executing remediation steps, while I handled comms to leadership.
- Using pre-prepared notes and runbooks, we doubled the heap size on the brokers, restarted them sequentially, and monitored consumer group rebalancing.
- I kept leadership informed with 15-minute updates so they didn't disrupt the technical team.
- After stabilization, I led a blameless post-mortem to capture what went wrong, and we discovered this was partly due to poor capacity planning and lack of automation.

Result

- Service was restored within 45 minutes, avoiding what could have been hours of downtime.
- In the weeks following, I drove systemic fixes:
 - Implemented **automated monitoring and self-healing scripts** for heap pressure.
 - Built **clearer runbooks** so future on-calls wouldn't rely on heroics.
- Personally, the experience gave me deep empathy for engineers being woken up at 3 AM. Today, as a senior EM, I channel that empathy into leadership — prioritizing automation, investing in psychological safety during incidents, and ensuring our abstractions reduce cognitive load rather than add to it.
- Eventual migration to Amazon MSK (managed Kafka)**

Follow up and takeaway

2. Was Kafka on-prem or managed?

- It was **self-managed on AWS EC2**. At that time, MSK wasn't mature enough for us. We owned the clusters, configs, and ops. Which meant it *was* our job to bring them back.

3. Why didn't first responders fix it themselves?

- The issue wasn't straightforward. Out-of-memory errors across all brokers required **deep familiarity with JVM tuning, heap sizing, and Kafka internals**.
- My pre-prepared notes + comfort with terminal multiplexing (tiled sessions across brokers) allowed me to act quickly and confidently where others hesitated.

4. How did you ensure no data loss?

- Restarted brokers **sequentially**, never all at once.
- Checked ISR before and after each restart → ensured replicas were healthy.
- Monitored consumer lag and rebalance events to validate recovery.

5. What did you change afterwards?

- Automated heap sizing + scaling policies.
 - Wrote formal runbooks and added them into the on-call playbook.
 - Advocated (successfully) for moving to MSK — reducing operational risk long-term.
-

▼ Balancing Abstraction vs Flexibility

Situation

"When I stepped in to lead the cloud platform team, the energy on the floor was... tense. Our developers were genuinely frustrated. Every time they wanted to ship a new feature, they had to fight a ticket queue just to get the basics: an **IAM role**, some **networking rules**, a **CI/CD pipeline**, or their **Kubernetes configs**.

Frankly, both sides felt blocked. Security felt they had to act as the gatekeepers to keep things safe, and the engineers felt they were being treated like children. The result was exactly what you'd expect: inconsistent standards across the org, a significant slowdown in delivery, and rising friction between teams."

The Mission: Designing the 'Golden Path'

"My immediate goal was to break that deadlock. We needed to shift the conversation from *'No, you can't do that'* to *'Here's the fastest, safest way to do it.'* The mission wasn't just to build a platform; it was to design a **'Golden Path'**—a fully paved road for the top 80% of use cases that made **security, compliance, and reliability non-negotiable**. If you followed the path, you got speed and autonomy.

Action: Automating Safety, Empowering Developers

"We tackled this from three core directions:

1. Defining the Path with Terraform Modules

"We looked at the 80%—the stateless APIs, the event consumers, the batch jobs—and my team built hardened, versioned **Terraform modules**. These weren't just templates; they were full solutions. When a developer used one, they instantly got all the guardrails baked in: **least-privilege IAM**, default **encryption**, robust **logging**, and mandatory **cost tagging**. It basically turned a complex, multi-step provisioning process into a single module call."

2. Enforcing Guardrails with Policy-as-Code

"We integrated **OPA** (Open Policy Agent) and **Sentinel** right into the CI/CD pipeline. This was the real game-changer. Security policy shifted from being a manual ticket review *after* the fact to an automated check *before* the code even merged. If a developer tried to use a wildcard IAM role or forgot a mandatory tag, the build failed instantly with a clear, actionable error message. No tickets, no waiting—just fast, direct feedback."

3. Enabling True Self-Service

"Finally, we wrapped everything into a simple **CLI tool** (think `svc init`). A developer could run one command, and in under an hour, they had a compliant, pre-wired repo with the right Terraform stack, a functioning pipeline, and monitoring hooks. The days of waiting 2–3 weeks for basic setup were over."

The Results: Guardrails, Not Gates

"We worked hard to frame this program as '**guardrails, not gates**,' driving adoption through empathy, not mandate. We ran office hours and constantly paired with early adopters to ensure the Developer Experience (DX) was smooth.

The outcomes were transformative:

- **Adoption soared:** Within three months, **85%** of all new services were being provisioned through the Golden Path.
- **Time to market plummeted:** Time to first production deploy went from weeks of back-and-forth tickets to **same-day** deploys.

- **Security post-incident review shrunk:** Incidents related to IAM and network misconfigurations dropped by about **60%**. Our CI/CD policy logs gave us all the evidence we needed, virtually eliminating cloud infra-related audit findings.
- **Culture shifted:** The easiest path became the safest path. Developers felt trusted because we gave them the keys to the kingdom, but with an automated seatbelt. And Security could finally focus on emergent threats instead of being the 'bad cop' of compliance."

✓ This rewrite gives:

- **Clear Situation/Tension** (dev speed vs. security control).
- **Concrete Actions** (Terraform, OPA, CLI, exceptions).
- **Leadership Framing** (empathy, "guardrails not gates").
- **Strong Measurable Outcomes** (adoption %, speed, incident reduction).

▼ Cloud Migration(Top-Down & Bottom-Up

Situation

At the bank, we faced a large-scale cloud migration mandate. Executives wanted **cost reduction, compliance, and agility**, but engineers were skeptical. They feared more bureaucracy, extra controls, and less autonomy. The risk was that without **alignment at both levels** — exec sponsorship and engineer adoption — the migration would stall or fail.

Task

As Senior EM, my role was to drive the transformation. I had to:

1. Secure **executive sponsorship** and align KPIs to business objectives.
2. Win **developer trust** by reducing friction and showing that compliance could be built into workflows, not bolted on.
3. Deliver migration progress at scale without paralyzing innovation.

Action

- **Top-Down Alignment (Execs & Directors):**

- Partnered with the CISO and platform director to define success metrics: % workloads cloud-ready, mean-time-to-deploy improvements, SLA compliance, and audit pass rates.
- Built these into the leadership scorecards so every director had visibility. This ensured cloud adoption was not “one team’s project,” but part of everyone’s performance objectives.
- **Bottom-Up Adoption (Engineers & Teams):**
 - Created a **developer portal** (based on Cortex-like principles) where teams could provision cloud infra via **Terraform-backed templates**.
 - Integrated **policy-as-code** guardrails into pipelines: no public S3, mandatory tags, IAM least-privilege, TLS enforced. Violations surfaced at PR time with clear error messages — no tickets.
 - Ran **roadshows and workshops** to demo workflows and capture pain points. Early adopter teams influenced templates, which gave them ownership.
- **Bridging the Two:**
 - Introduced **team-level scorecards** (compliance, cost hygiene, deployment lead time). These were not punitive, but transparent and discussed in **weekly business reviews** across product and platform leaders.
 - Escalated blockers to executives when teams lacked resources or approvals — exec “laser beam” focus cleared those obstacles.

Result

- **Adoption:** Migrated 70% of targeted workloads in the first 12 months, ahead of schedule.
- **Compliance:** Achieved zero high-severity audit findings in cloud workloads; policy logs became evidence for regulators.
- **Velocity:** Median time to deploy new services dropped from 2–3 weeks to under 1 day.
- **Culture:** Engineers saw security as built-in, not bolted on. Execs had dashboards that tied adoption to business goals. The cultural narrative

shifted from “migration mandate” to “developer empowerment.”

- **Personal Growth:** This experience taught me that **cloud transformation is 50% technology, 50% culture**. Top-down sponsorship provides air cover, but bottom-up adoption is what makes change sustainable.
-

Likely Follow-Up Questions

- *“How did you convince execs to back your KPIs?”* → talk about linking KPIs to business risk (audit penalties, infra spend) and business opportunity (faster product launches).
 - *“What resistance did you face from engineers?”* → describe initial pushback, and how you turned critics into co-creators of the templates.
 - *“How did you measure cultural adoption, not just workload numbers?”* → adoption % + reduced exceptions + developer NPS/feedback.
 - *“What would you do differently?”* → highlight that you’d invest earlier in change champions or embed platform engineers in product teams.
-

Where It Fits in Your Framework

- **Leadership & People** → aligning two very different stakeholder groups.
 - **Execution & Delivery** → migrating workloads at scale.
 - **Technical Judgement** → building portal, policy-as-code, scorecards.
 - **Impact & Outcomes** → measurable adoption, audit results, cultural change.
-

 This version now shows:

- **Strategic leadership** (exec sponsorship + cultural adoption).
- **Tactical execution** (developer portal, policy-as-code).
- **Measurable impact** (70% migration, zero audit findings, velocity gains).
- **Leadership maturity** (change management + culture).

▼ SBOM Vulnerability Management

Situation

When I stepped into the role, there was friction between engineering and the CISO-led central security team. Developers were receiving vulnerability alerts they couldn't action—mostly from base images or shared libraries they didn't own. This created alert fatigue, wasted effort, and broke trust. Despite the volume of alerts, our risk posture wasn't improving.

Task

Build a solution that would:

- Taking 45 days on average to remediate vulnerabilities
- Cut through alert noise and make vulnerabilities actionable.
- Clarify ownership across platform and app teams.
- **Quantify and demonstrably improve our security posture** without slowing delivery.

Action:

- Reframed the problem from “send all alerts” to deliver vulnerability intelligence as a service”
- Aligned stakeholder(BOW): partnered with the CISO, platform leads, and other EM's to ensure shared understanding and buy-in
- Built a streaming graph-based ownership model. I designed a streaming application that
 - Consumed the raw SBOMs generated by the CISO's scanners from our central Kafka topic
 - Enriched the data in real-time by cross-referencing it with our internal application catalog (to map services to teams) and our container registry metadata (to distinguish base image layers from application layers)
 - Modeled this enriched data in MongoDB. We opted for a document model to represent the graph-like relationships (CVE → Layer → Application → Owner) as it allowed for rapid application development. This was a pragmatic choice to avoid the steep learning

curve of a dedicated graph database, which we didn't have time for. This gave us persistent, queryable map connecting CVEs to their true owners.

- Automated ticket routing based on the graph:
 - If a CVE was traced to a base image layer → an aggregated ticket was auto-assigned to the platform team
 - If a CVE was in an application layer → a targeted Jira was auto-added directly to the owning app team.

Result

This platform transformed security from a blocker into an automated service.

- **Reduced unactionable vulnerability alerts to app teams by 92%.**
- **Decreased mean-time-to-remediate (MTTR) for critical base image vulnerabilities from 45 days to 6 days**, as the Platform team could now patch centrally.
- **Achieved 99% automated ownership mapping** for all new vulnerabilities, virtually eliminating triage overhead.
- We used this data to give the CISO the first-ever accurate dashboard of our true risk posture, rebuilding trust between security and engineering.

Follow up

What do you mean by graph model

"Good question — by graph-based model, I mean we represented our software supply chain and dependencies as a directed graph. Each node was a software component — like a base image, a shared library, or an app — and the edges represented dependencies between them. This allowed us to trace any vulnerability (like a CVE) back to its true origin and owner. So instead of treating every alert equally, we could say, 'This CVE exists in a base image

used by 15 apps — the platform team owns it.' That graph let us automate ticket routing and reduce noise massively."

"We initially used MongoDB to store our components and their dependencies, since our graph was relatively shallow and we already had Mongo in our platform stack. Each document represented a node, and we modeled edges using array fields like `dependencies`. For deeper analysis and traversal, we later considered migrating to Neo4j, but in most cases, our use case didn't require heavy graph operations — so Mongo with precomputed relationships was sufficient."

▼ Scaling Teams

Situation

When our organization doubled in size within a year (from about 25 to 50 engineers), we weren't allowed to add formal managers — only engineers and contractors. That created an immediate leadership gap: delivery needed structure and alignment, but hierarchy wasn't an option.

Task

I needed to scale our delivery model without adding management layers — creating **clarity, accountability, and autonomy** across multiple teams. The goal was to keep velocity high and avoid chaos as the org grew.

Action

- **Created a Tech-Lead Operating Model:** I identified 5 senior engineers as **tech leads**, each owning a domain (data platform, cloud infra, ingestion, observability, developer tooling). They became "player-coaches" — accountable for delivery within their stream.
- **Defined Clear Swimlanes:** Mapped ownership boundaries and interfaces between streams so contractors and ICs could execute independently with minimal handoffs.
- **Established Lightweight Rituals:** Introduced a bi-weekly **Tech Leads Forum** where leads aligned on priorities, dependencies, and resourcing —

effectively functioning as a leadership team.

- **Scaled Communication:** I met weekly with each lead to coach on stakeholder management, estimation, and conflict resolution — the soft skills usually handled by managers.
- **Visibility Without Bureaucracy:** Used dashboards and shared OKRs to track delivery metrics, freeing me from micromanaging and giving leads full ownership.

Result

- Delivery capacity scaled by **2x in under 6 months**, with zero new management overhead.
- **Tech leads matured into strong servant leaders** — one later transitioned into an official EM role once the org policy changed.
- **Contractor output improved** significantly because communication flowed through empowered leads rather than waiting for top-down direction.
- The model became a **blueprint for distributed leadership**, later adopted by two other teams in adjacent orgs.

Reflection

This experience taught me that leadership isn't about titles — it's about **creating a system where accountability, decision-making, and trust scale faster than hierarchy.**

▼ Vision / Future of Cloud Ops

| Situation

We had a strategic mandate to modernize infrastructure (VMs to Containers, Dedicated to Shared) but adoption was slow. Engineering teams treated infrastructure as “free” because they lacked visibility into financial trade-offs of their architectural choices..

| Task

I needed to shift cost accountability left. My goal was to build a “What-If” simulation engine that allowed engineers to quantify the impact of migration

before writing a line of code, without needing a finance degree.

| Action

- **Data Strategy:** I partnered with platform engineering to ingest full usage metrics and normalize them against our cloud provider rate cards and internal chargeback models
- **The Engine:** We built a projection engine that moved beyond simple reporting. It allowed for variable substitution - teams could plug in their current throughput and see a side-by-side P&L of "As-Is" vs "Shared Cluster" architecture
- **The UX:** I integrated this directly into the developer portal, ensuring the feedback loop was instant and self-serve"

| Result

- **Financial:** Identified **\$12.5M in annualized opportunities in the first quarter.**
- **Cultural:** "It changed the conversation. Cost became an engineering constraint we optimized for, just like latency or availability"

▼ Improved communication with stakeholders - 20% (need to figure out measurement)

Action:

- **Mapped and prioritized stakeholders:** I led brainstorming sessions to identify everyone affected by our work, from product and design to legal and customer success. We documented their interests and influence in a stakeholder register, then used a power/interest grid to prioritise engagement strategies.
- **Clarified responsibilities:** We co-created a RACI matrix for our next big initiative, defining exactly who was Responsible, Accountable, Consulted and Informed for each deliverable. This eliminated duplicated work and ensured no task lacked an owner.
- For feature prioritisation we used RICE scoring and impact-effort matrices. The team decomposed features for accurate effort estimates, and I facilitated trade-off discussions with product and design. We also

implemented Meta's 2-2-2 code review method to speed up reviews and adopted TDD and CI to get rapid feedback from our own sy

Situation: When I first joined my current group, I stepped in mid-project to find a chaotic environment. Communication with business partners was broken, they weren't receiving updates and progress felt opaque. internally, the engineering team lacked clarity and direction in many ways as well. Things were basically stuck.

Action : - Split into three-pronged approach

Stakeholder alignment - I started by conducting a stakeholder mapping approach - high power/low interest - to tailor cadence in depth. This allowed us to know how much detail each stakeholder require based on where they fall

Internal Clarity - To solve the confusion around ownership, I introduced a RACI (Responsible, Accountable, Consulted, and Informed) model. I facilitated workshops with our team and partner function to map our every key deliverable, assigning a clear responsible and accountable party for each. Immediately eliminated ambiguity

Engineering Velocity:- I identified our code review process as a major bottleneck. to reduce this friction, I implemented Meta's 2-2-2 review method: limit PR to 200 line, requiring no more than 2 reviewers and establishing a 2-day turnaround expectation.

Result —

These actions led to significant and measurable improvements. We succesfullly cut our delivery cycle by 35%. More importantly, we reestablished trust and confidence with our business stakeholders. Internally as well, the teams morale and exeecution spped increased.

▼ Inclusive Leadership & diversity

The initial team I picked up was one giant team, participation in design and decision-making was done by few of the senior members

- Task

Situation

In a globally distributed org, I noticed quieter engineers — often from underrepresented groups — struggled to have their voices heard in design discussions. This risked us missing good ideas and hurting inclusivity.

Task

I wanted to foster a **truly inclusive environment** where all engineers felt ownership and psychological safety.

Action

- Instituted **rotating meeting leads** so not just the loudest voices ran discussions.
- Shifted design reviews to include **written async input**, giving global team members equal opportunity.
- Partnered with HR to add **diverse interview panels** and anonymized resume screening for fairness.
- Created mentorship pairings for junior engineers from underrepresented groups with senior ICs.

Result

- Team surveys showed a 25% increase in "I feel my opinions are valued."
- Diverse hiring improved year-over-year.
- Retention improved, and quieter engineers started driving key features.
- The org built a reputation internally as a "safe, inclusive, high-performance" culture.

▼ How do you evaluate a candidate for tech and culture fit

For technical fit, I look for problem-solving depth, not just specific tools — can they reason about scale, tradeoffs, and design? For cultural fit, I don't define it as 'being like us'; I look for alignment with values like collaboration, accountability, and openness. I use structured interviews with scenario-based questions and I involve diverse panel members to reduce bias. That way, I hire people who raise the bar technically while strengthening team culture."

Situation: *At Citi, we needed to scale our data engineering team quickly — but previous hiring had been inconsistent and led to gaps in skills and alignment.*

Task: *I needed to create a hiring approach that ensured strong technical bar and inclusive culture.*

Action:

- *I introduced a structured hiring loop: system design for scale, coding for fundamentals, and a values interview focused on collaboration and ownership.*
- *I trained interviewers on structured scoring rubrics to avoid "gut feel."*
- *I emphasized diversity: we widened sourcing pools, ensured balanced panels, and assessed for inclusivity in answers (e.g., how candidates collaborate in distributed teams).*

Result: *We built a 25+ engineer org that was both technically strong and diverse — with a 90%+ offer acceptance rate. The mix of perspectives directly improved design discussions and innovation velocity.*

▼ Managing an under performer Improvement

Hook

- Used a 60-day plan + senior pairing to turn around performance and improve reliability.

S (Situation)

- A mid-level engineer was underperforming (missing sprint goals, code failing QA).

- The engineer seemed disengaged.
- This was impacting team morale and delivery reliability.

T (Task)

- Help the engineer improve their performance and regain confidence.
- **Constraint:** Must be done without creating a culture of fear or micromanagement.

A (Action)

- Held a candid 1:1; identified the root cause: **low confidence** and **unclear expectations**.
- **Co-created a 60-day improvement plan** with specific, measurable goals.
- **Paired the engineer with a senior mentor** for code review and design help.
- Established **weekly check-ins** for real-time feedback.
- **Celebrated small wins** publicly to rebuild their confidence.

R (Result)

- **Metric:** The engineer's defect rate dropped by **35%**.
- **Observation:** Delivery reliability improved noticeably within two sprints.
- **Engineer Impact:** Regained confidence and later **owned a core service redesign**.
- **Team Impact:** Reinforced the team's trust in a coaching-based leadership style.

▼ How to deal with ambiguity

STAR Story: Application Dependencies — Turning Ambiguity into Alignment

Situation

I was asked to lead an initiative called **Application Dependencies** after our **multi-million-dollar cloud migration program stalled**.

We couldn't safely plan migration waves because there was **no reliable view of how applications depended on one another**.

The brief I received was simply: *"We need to understand our dependencies."*

Once I dug in, the ambiguity ran deep — there was **no single definition of "dependency," no owner, and no reliable data source**.

Each stakeholder group saw it differently:

- **Infrastructure teams** viewed dependencies as compute or hosting relationships.
- **Application owners** thought in terms of upstream and downstream data flows.
- **Risk teams** focused on recovery chains and attestation.

Data lived across **CMDBs, spreadsheets, and recovery catalogs**, often outdated or duplicated — resulting in **conflicting reports, broken accountability, and no trusted view during audits or migration planning**.

Task

I reframed the goal from *"draw a map"* to:

“Build a single, trusted, and sustainable platform for dependency data.”

That meant defining a **common language**, reconciling **data across multiple systems**, and building a **scalable UI and attestation workflow** usable by architects, app owners, and risk teams.

Action

1. Discovery & Alignment

I began with **cross-functional workshops** across architecture, infrastructure, and risk teams to answer one core question:

“When you say ‘dependency,’ what decision are you trying to make with it?”

This surfaced **three dependency layers** — application, infrastructure, and recovery.

Working with CMDB and portfolio teams, I defined a **unified data model and taxonomy** that captured all three consistently — providing the foundation for execution.

2. Structuring Delivery

Once definitions and scope were clear, I created a **Jira Initiative** and broke it into **four Epics**:

1. **Data Standardization** – unify schema and reconcile sources.
2. **Visualization Layer** – build APIs and interactive graph views.
3. **Attestation Workflow** – embed validation and ownership.
4. **Governance & Reporting** – define metrics and controls.

Each Epic was owned by a **dedicated pod (4–6 engineers)** led by a **Tech Lead**.

I used **RACI** to clarify responsibilities, held **bi-weekly cross-pod syncs**, and tracked progress in **Jira dashboards** tied to business outcomes like data accuracy and audit readiness.

3. Engineering Execution

We initially explored **Neo4j** for its graph capabilities, but a POC revealed steep operational overhead and limited in-house expertise.

I led a pivot to a **Java-based streaming API** with a **JavaScript graph visualization**, which progressively loaded nodes for performance and scalability — all within our existing tech stack.

4. Governance & Adoption

We embedded a **quarterly attestation workflow** where app owners validated or retired dependencies, ensuring continuous data quality and clear accountability loops.

Result

- Delivered a **trusted system of record** for dependencies — unblocking the **cloud migration program** that triggered the initiative.
- Unified **four fragmented data sources** into one authoritative platform.

- **Reduced attestation effort by 40%.**
 - Adopted by **1,300+ critical applications** globally.
 - **Architects** gained real-time impact analysis for infra changes; **risk teams** halved audit prep time.
-

Reflection

"In complex, ambiguous programs, alignment comes before architecture — once everyone defines the same 'why,' structure and delivery become straightforward."

- ▼ Tell me about a time you had to approach breaking down complex project. E
- .NET TIBCO EMS Migration (Ownership lifecycle approvals, onboarding, transfer, retirement, dependency classification)

Situation

At Citi, I led the modernization of our enterprise approval workflow system, which was running on an aging **.NET and TIBCO EMS** stack in our private cloud. The platform was unstable, nearing end-of-life, and the bank had a **global directive to retire TIBCO** and standardize on other preferred solutions.

Because the workflow ecosystem was deeply interconnected across multiple business lines, a **big-bang rewrite** wasn't feasible — we needed a structured, incremental approach that avoided business disruption.

Task:

I was responsible for **designing the technical and delivery approach** to break down this large, high-risk migration into clear, independently deliverable components. The solution had to:

- Enable **zero-downtime migration**,
 - Satisfy **compliance and audit** requirements, and
 - Allow **multiple teams to work in parallel** without stepping on each other.
-

Action:

To manage both technical and delivery complexity, I combined **architectural decomposition** with **structured execution planning**.

- I began with **domain and capability mapping workshops** to identify the system's core functions — submission, routing, notifications, and audit logging — and their dependencies.
- Architecturally, I adopted a **Strangler Fig approach** and selected **Solace PubSub+** as our **hybrid event backbone**, deployed across both **private cloud and AWS EKS**. This gave us guaranteed delivery, request/reply, and hybrid cloud connectivity to keep legacy and new components running side by side.
- We implemented an **API façade and message bridge** in front of TIBCO, allowing us to route events to both systems while gradually migrating functionality to new **Java microservices** on AWS.
- To validate early, I led the creation of a **"walking skeleton"** — a minimal end-to-end approval flow from request to confirmation — before expanding to more complex cases.
- Once the architecture was proven, I formalized the delivery structure in **Jira**. I created a **Jira initiative** for the modernization program and broke it into **epics aligned with each vertical slice** — submission, routing, notification, audit, and platform services.
 - Each epic had its own backlog, ownership, and CI/CD pipeline, enabling teams to **build and test independently**.
 - We maintained a **dependency matrix** to sequence integrations and prevent blockers (e.g., ensuring queues and Solace topics were provisioned before consumers went live).
- Everything — infrastructure, brokers, queues, ACLs — was codified in **Terraform and Helm**, giving us repeatable, auditable deployments.

Result:

- Delivered the **first AWS-backed approval flow within 12 weeks**, meeting end-of-life and compliance deadlines with **zero downtime**.
- Reduced incidents by **over 50%** and improved approval latency by ~30%.

- Enabled **parallel team delivery** through the Jira initiative/epic structure, eliminating cross-team blockers.
- Established a **repeatable cloud-migration blueprint** now used across multiple modernization programs in the bank.

How did you ensure no data loss during migration

"We used Solace persistent queues with built-in durability so no messages were lost even during cutovers.

We set clear retention and expiry policies — long enough for retries and audits, but not treating the broker like a database.

Any failed messages went to a dead-letter queue, and we had replay enabled for forensics and recovery.

In short, we designed for guaranteed delivery and traceability, but kept Solace focused on messaging, not long-term storage."

How did you codify

"We wanted every part of the environment — from infrastructure to messaging — to be reproducible and auditable, so we codified it.

Using **Terraform**, we provisioned all AWS and private-cloud resources — VPCs, EKS clusters, load balancers, IAM, storage, even Solace broker instances.

Then for the application and Solace layer, we used **Helm charts** to manage broker configuration: creation of **Message VPNs, queues, topic endpoints, ACLs, client profiles**, and environment variables for each namespace.

Everything lived in **Git**, and deployments ran through **CI/CD pipelines** — meaning any change (like a new queue or ACL) had a pull request, review, and audit trail.

This made environments consistent across dev, QA, and prod — and gave us a full compliance record of who changed what, when, and why."

🧩 Deeper technical detail (if interviewer wants more)

Layer	Tool	Description
Infrastructure (AWS / Private Cloud)	Terraform	Provisioned EKS clusters, subnets, security groups, VPC peering, Direct Connect, RDS, S3, IAM roles, CloudWatch metrics, and Solace brokers (AMI or container).
Solace configuration	Helm + Solace Config Sync / SEMP API	Managed creation of Message VPNs, queues, topic-to-queue bindings, ACL profiles, and client usernames. Config files parameterized per environment.
Microservices	Helm Charts	Deployed Java Spring Boot services with configs (topics, subscriptions, ACLs). Charts versioned per vertical slice (e.g., submission, routing, audit).
Pipelines	GitHub Actions / Jenkins / Argo CD	Automated plan → review → apply → verify. Pipeline executed Terraform plan, Helm upgrade, and post-deploy validation (queue health, message replay test).
Auditability	Git + CI logs + change tickets	Every config change PR tied to a Jira ticket. Pipeline artifacts and Terraform state stored centrally (S3 + DynamoDB) for traceability.

9 "How did you measure success?"

"We defined it in three dimensions:

- (1) Business continuity — zero downtime,
- (2) Operational metrics — 50% fewer incidents, 30% faster approvals,
- (3) Organizational leverage — other teams reused the blueprint.

Those became our success KPIs."

👉 *Why they ask:* To test outcome orientation.

👉 *Signal to send:* You track tangible metrics, not just delivery dates.

🚀 10 "What was your role vs. your team's?"

"I owned the overall technical direction and delivery strategy — setting the migration pattern, defining the Jira structure, sequencing cutovers, and unblocking teams.

Tech leads owned each vertical slice and implementation.

My focus was on enabling velocity and predictable delivery."

▼ Tell me about a complex project you worked on (DRIFT)

▼ L2 Rollups at mastercard

▼ Cloud agnostic infrastructure at Citi

When Citi entered a major strategic partnership to expand its footprint on GCP, my organization was already heavily invested in AWS. The change created uncertainty — multiple teams had ongoing programs, AWS-native tooling, and regulatory deadlines tied to existing environments.

I led an initiative to define a consistent deployment model that would work across both clouds, reducing rework and dependency risk. Working with our cloud platform and governance teams, I proposed using Terraform modules and policy-as-code guardrails to abstract provider differences.

My team built the first end-to-end reference implementation, validated it with security and compliance, and onboarded early adopters from three lines of business. This became the blueprint for subsequent migrations and positioned our org to move workloads between clouds without vendor lock-in.

In the end, we reduced provisioning lead time by ~40% for new projects and created a reusable pattern later adopted by other internal cloud teams.

▼ Modernizing DRIFT into a REAL-Time Data Fabric"

Ingests tons of data - GEM, ACM, Cyberark, ConfigDb, CICD

Situation

"In my current role, I one of the initiative I lead is DRIFT — our enterprise data fabric that ingests around 2 billion records a day from nearly 300

systems.

It historically ran on a Hadoop + Hive + Spark setup — reliable, but entirely batch-oriented.

The pain was that policy checks, reconciliation, and data quality feedback came hours after ingestion — far too slow for operational or compliance use cases.

my task was to make this architecture more **real-time, governed, and resilient**, without disrupting the existing ecosystem.”

Task

“My goal was to design and lead the implementation of a new streaming curation layer that:

- Processes data continuously instead of nightly,
- Maintains lineage and data contracts end-to-end,
- And guarantees data correctness under high volume — billions of events per day.

We wanted to achieve this by integrating **Kafka, Flink, and Iceberg**, while still coexisting with our Hadoop infrastructure.”

Action

“The first step was proving that we could replace nightly Spark ETL with stateful Flink jobs that validated and enriched data in flight.

I partnered with platform engineers to build:

- A **schema and contract validation framework** on top of the Schema Registry,
- **Stateful deduplication** and **late-event handling** in Flink,
- And **PII masking and policy checks** embedded using OPA.

For the curated zone, I introduced **Apache Iceberg** to replace static Hive tables — this gave us ACID guarantees and time-travel.

We piloted with three volatile domains — identity, infrastructure, and entitlement data — processing about 300 million daily events initially.

The migration involved new DevOps practices: checkpointing, RocksDB tuning, Flink job savepoints, and real-time observability through Prometheus and Grafana.

I led code reviews, incident runbooks, and established golden-path templates so teams could onboard safely."

Result

"After a few painful iterations, we achieved end-to-end exactly-once streaming ingestion with near-real-time curation.

Data that previously took 4–6 hours to land in curated Hive tables now becomes queryable in under a minute.

Quality metrics are published live, and every dataset has versioned lineage via Iceberg snapshots.

The pilot's success made this our new standard pattern, and it's now expanding across multiple domains.

Just as importantly, the team learned how to manage long-lived streaming jobs — monitoring RocksDB state, handling schema evolution safely, and automating recovery.

It was technically the hardest thing I've done recently, but it's also the most rewarding — it changed how the company thinks about data freshness and governance."

▼ DRIFT for Senior Leader

Ingests tons of data - GEM, ACM, Cyberark, ConfigDb, CICD

Situation

"In my current role, I lead the 'DRIFT' enterprise data fabric, which ingests around 2 billion records a day from nearly 300 different systems.

When I took over, the platform was technically functional but strategically failing. It was a complex mix of technologies:

- **A reliable Hadoop/Spark/Hive stack** for batch analytics.
- **A chaotic ingestion landscape:** decades-old SFTP file-drops, nightly DB-to-DB syncs, and intermittent API endpoints.

The core business problem was **risk**. All these ingestion methods fed into a batch process. This meant our data quality, reconciliation, and—most critically—our **security and compliance policy checks** ran **4-6 hours late**. For detecting a compromised account or a toxic access combination, a 6-hour delay was an unacceptable exposure for the firm."

Task

"My mandate was to lead the initiative to fix this. The goal was to re-architect our platform to make our data **real-time, governed, and resilient**.

This wasn't just a technical upgrade; it was a multi-faceted strategic objective:

1. **Reduce Business Risk:** Cut data-to-action latency from hours to under a minute.
2. **Manage Complexity:** Tame the 300-source ingestion chaos without a 2-year, high-risk "rip and replace."
3. **Future-Proof the Platform:** Implement a modern, scalable architecture that could coexist with our massive existing investment in Spark."

Action

"My strategy was two-pronged: one for the organization and one for the technology.

1. The Organizational Strategy: 'Coalition & Pave'

First, I had to manage the organizational friction.

- **'Coalition':** I knew I couldn't get 300 teams to change at once. I started by partnering with the **'coalition of the willing'**—our 25% most critical and modern sources, like Security and Infrastructure. I pitched them on solving *their* business need (real-time alerts), and they became our champions.

- **'Pave':** For the other 75%—the legacy SFTP and DB syncs—I had my team **absorb the complexity**. We built a "paved road" (our 'pragmatic bridge') that turned their batch files and DB changes into event streams using CDC and an event-driven file shredder. We made the 'right way' the 'easy way' for them, which let us move fast.

2. The Technical Strategy: 'Unify & Coexist'

- **Unify:** I mandated **Kafka** as the single 'front door' for all data. Our CDC, file-shredder, and modern microservices *all* published to Kafka. This decoupled our complex sources from our new processing logic.
- **Coexist:** This was the most critical decision. We built the new real-time 'hot path' with **Flink**, chosen for its true streaming and robust state management. But we deliberately used **Apache Iceberg** as our sink. Iceberg was the 'peace treaty' that allowed our new Flink jobs (for real-time writes) to **coexist** with our existing, massive **Spark** investment (for BI queries and batch maintenance). We didn't throw away our Spark assets; we refocused them.
- **Build for 'Day 2' Operations:** To make this sustainable, we embedded **OPA** for policy checks but used **Flink's Broadcast State** to update those policies with *zero downtime*. This ensured my team didn't become a bottleneck for the compliance department."

Result

"This strategy was transformative.

- **Business Risk Reduced:** We cut our data latency from **4-6 hours to under 1 minute**. This means we now detect and can act on security threats and compliance breaches in near-real-time.
- **Platform Modernized:** We now have a single, unified, and governed platform. The 'hot path' handles our real-time needs, while our 'cold path' of Spark jobs still runs reliably for BI and reporting, often reading from the *same* Iceberg tables Flink writes to.
- **Team Upskilled:** I successfully led my team's evolution from batch-oriented engineers to experts in managing long-lived, stateful streaming

applications, handling everything from RocksDB state to Iceberg compaction. We delivered a step-change in capability without ever disrupting the existing business."

▼ DRIFT - New Yahoo

Yahoo-Optimized Execution & Leadership Story

HEADLINE (20 seconds)

"One of the most complex delivery efforts I've led was modernizing our enterprise data fabric, DRIFT — a platform ingesting ~2 billion records a day from nearly 300 legacy and modern systems. I led the architecture, team transformation, and cross-org alignment that cut data-to-action latency from 6 hours to under 1 minute, enabling near-real-time security and compliance intelligence across the company."

S — Situation

When I took over DRIFT, it was technically functional but strategically failing.

We were ingesting **two billion records a day** from:

- Modern APIs
- 20-year-old SFTP drops
- Nightly DB-to-DB replication
- Intermittent, undocumented feeds

All this landed in a batch Hadoop/Spark environment that ran **hours behind real time**.

The business risk was huge:

- **Security alerts detected 4–6 hours late**
- **Toxic access combinations evaluated long after they occurred**
- **Compliance policy checks running on stale data**

In one Q1 governance meeting in building 4, our CISO (let's call him *Mark*) said plainly:

“We are operating blind for hours every day. We need real-time intelligence, not forensic reporting.”

I was asked to lead the turnaround — architecturally, organizationally, and from an execution standpoint.

T — Task

My mandate was to **re-architect DRIFT** into a platform that was:

1. **Real-time** — reducing latency from hours to <1 minute
2. **Governed** — enforcing compliance & PII policies in-flight
3. **Reliable** — able to handle billions of heterogeneous events daily
4. **Backwards-compatible** — without disrupting hundreds of dependent Spark workloads
5. **Organizationally adoptable** — across 300+ data-owning teams

I needed to drive this transformation with my **team of platform, data, and streaming engineers**, while aligning Security, Compliance, Infrastructure, and Data Engineering.

A — Action

1. Created a dual-track strategy: “Coalition & Pave”

I quickly realized the hardest part wasn't technology — it was organizational inertia.

Coalition — win with the teams who cared most.

I partnered first with the teams that *needed* real-time insights:

Security, Identity, Infrastructure.

Their urgency created momentum and internal sponsorship.

Mark, the Security architect, became my first champion.

Pave — remove friction for everyone else.

For the 75% of legacy systems that couldn't change:

- I had my team **absorb their complexity**, not force theirs to modernize.
- We built a “pragmatic bridge”:
 - A CDC pipeline to turn DB changes into events
 - A file-shredder to convert SFTP drops into Kafka streams

We made the “right way” the **easy way**, eliminating political blockers.

2. Led the architecture redesign: “Unify & Coexist”

Technical direction was entirely my responsibility. I made three foundational decisions:

(a) Kafka as the single front-door

I mandated that **all** ingestion patterns — modern APIs, SFTP, DB copies — emit to Kafka.

This decoupled messy sources from the new processing path.

(b) Flink for the real-time hot path

I chose **Apache Flink** over Spark Structured Streaming because:

- Better state management
- True streaming, not micro-batch
- Millisecond-latency
- Exactly-once semantics guaranteed at scale

(c) Iceberg as the peace treaty

To avoid a risky rip-and-replace, I introduced **Apache Iceberg** as the unified storage layer.

This allowed:

- Flink to write real-time events
- Spark to continue running downstream analytics without breaking

This decision alone removed **years** of potential migration risk.

3. Built governance into the streaming path

I embedded compliance logic directly into Flink using:

- **OPA policy enforcement**
- **Broadcast state updates with zero downtime**
- **In-flight PII checks**

This gave Compliance full control without engineering bottlenecks.

4. Elevated the team and culture

This transformation required significant upskilling.

How I led my team:

- I paired senior engineers with me for design reviews and state management decisions.
- I coached two engineers (let's call them *Rahul* and *Emma*) from batch-thinking to streaming experts.
- I instituted "streaming incident game days" to build confidence with state restoration, checkpointing, and recovery scenarios.
- I created a rotational on-call model to improve ownership across the team.

Within 6 months:

- Rahul became the de-facto expert on Iceberg compaction strategies
- Emma led our adoption of RocksDB state backends and was promoted

The team became **self-sufficient**, not dependent on me.

5. Drove adoption across 300 teams

I ran weekly office hours and integration clinics with the system owners.

To reduce resistance, I provided:

- Pre-built ingestion templates
- Kafka onboarding scripts
- Change-impact analysis for each team
- Dashboards showing their latency improvements

By removing friction and focusing on value, adoption accelerated organically.

R — Result

★ Cut latency from 4–6 hours → under 1 minute

Near-real-time visibility for Security & Compliance.

★ Unified platform serving both real-time and batch

Flink + Iceberg + Spark coexisted seamlessly.

★ Improved risk detection and reduced exposure window

Security incidents surfaced **within the same minute** they occurred — a game changer.

★ Stable, highly governed data flow

PII, policy checks, lineage, and provenance were applied in-stream.

★ Transformed the engineering team

Engineers grew from batch operators to experts in long-lived stateful streaming systems.

★ Zero disruption to existing downstream consumers

All Spark-based ETL and analytics jobs continued to function without refactoring.

★ Repeatable, scalable ingestion framework

300 system owners onboarded without redesigning their systems.

- Likely deep dive:
 - What made it technically hard or otherwise
 - Processing almost 2 billion events per day from nearly 300 heterogeneous sources, from modern APIs to 20 year old SFTP jobs - ensuring correctness under scale-deduping, late event handling, enforcing PII policies in-flight.
 - Traditional Spark micro-batches worked, but under spiky load they caused stair-steep latency and brittle recovery.
 - Guaranteeing “exactly -once” semantics at this scale was technically hard.
 - Architectural Challenge -
 - How to make both Spark and Flink setup co-exist was another thing.
 - Organisational Challenge -
 - The hardest non-technical challenge was organizational.
 - Many source systems were owned by different teams - some hadn't touched their integrations in years
-

“It was challenging on three fronts.

Technically, we had to guarantee correctness for billions of records daily across 300 inconsistent sources — mastering state, late events, and PII enforcement in real time.

Architecturally, we had to modernize without breaking the old world — building a Flink-based streaming path that coexisted with Spark through Iceberg. And

organizationally, I had to rally 300 data owners behind a change that touched every integration pattern they had.

Balancing those dimensions — correctness, coexistence, and change — was what made it both the hardest and the most rewarding project I've led."

▼ SCOUT → ME

★ STAR Story — SCOUT: Building a Unified ML-Powered Reliability Intelligence Platform

HEADLINE(20 seconds)

One of the most impactful project for me was SCOUT, an ML-driven incident-intelligence platform that analyzed millions of operational events a month, reduced repeated incidents by 33%, and cut triage time by over 40%. It turned unstructured ticket data into proactive reliability insights at scale.

S — Situation

2. "We were receiving millions of operational records a month across ServiceNow, CAI workflows, and other app applications. Each used different formats and most of the critical information lived in messy free-text. Because the data was fragmented, we had no reliable way to see patterns, correlate incidents with changes, or spot recurring issues. We needed a single intelligence layer to turn that unstructured data into proactive reliability insight."

We were drowning in millions of monthly operational records—each system (ServiceNow, CAI, others) used different formats, and the critical information was buried in free-text fields. there was no reliable way to correlate incidents, spot patterns, or act proactively.

T — Task

My goal was to lead the end-to-end design, development, and rollout of a platform that:

1. **Unified all operational data** from multiple sources into one intelligence layer
2. **Apply ML/NLP** to classify unstructured text and extract signals
3. Delivered team-specific, personalized reliability insights
4. Build a **cloud-ready architecture** that could scale, retrain, and support trend analysis
5. Deliver a platform that would **reduce repeated failures and operational risk**

The challenge was both technical and organizational: no single team owned all the data or the workflow.

A — Action

"First, I established the data foundation. I directed the effort to normalize our fragmented ServiceNow and CAI records into a single schema—because without clean, consistent data, any ML we applied would have produced noise instead of insight."

"Second, I steered the architecture toward an AWS-native, serverless design and made the explicit trade-off to use SageMaker's managed training and hosting. That decision reduced our MLOps burden dramatically and let the team focus their time on improving model quality rather than maintaining infrastructure."

"For the ML engine itself, I set the constraints. The team experimented with multiple model families, but I enforced a strict 110ms latency budget to ensure real-time triage. We landed on an ensemble meta-selector that maximized precision and recall without violating that performance requirement."

Finally, I treated internal engineering as a customer. I designed a 'relevance feed' rather than a generic dashboard, ensuring teams only saw insights tied to their specific stack. I then partnered with SRE leads to embed these insights into their weekly operational reviews to guarantee adoption

R — Result

1. Operational Efficiency:

"We cut triage time by 55%, effectively reclaiming thousands of engineering hours annually. By automating the noise, teams stopped manual data-hunting and focused on building."

2. System Stability:

"We achieved a **36% reduction in recurring incidents**. The platform surfaced hidden patterns—like seasonal auth loops—that allowed us to shift from reactive firefighting to proactive reliability engineering."

3. Technical Success:

"The meta-model approach drove precision from **68% to 89%**. This high accuracy built the trust required for adoption, giving leadership a single, real-time view of operational risk for the first time."

▼ SCOUT - Helped

S — Situation

"We kicked off SCOUT — an NLP-based incident clustering system. They owned model development, but they were blocked on:

- inconsistent operational data (CAI + SNOW)
- unclear taxonomy
- cloud architecture uncertainty
- noisy and unlabeled incident text
- lack of a clean training dataset

T — Task

"My role was to:

1. **Unblock the team** on data ingestion, normalization, and architecture.
 2. Use the opportunity to **lean into my data science background** to help shape the ML approach — without being the primary model builder.
 3. Help the team build something that was both **cloud-ready** and **business-aligned**."
-

A — Action

1. Activated my DS knowledge to sharpen the ML problem framing

Using my data science background, I clarified the model goal:

"Focus on reducing noise from inconsistent tagging by clustering semantically similar incidents — not predicting categories we know are already unreliable."

I helped the team narrow to 3 ML outcomes:

- semantic embeddings
- clustering for theme discovery
- a lightweight classifier for routing

This avoided overfitting, over-architecting, and unnecessary LLM complexity.

2. Partnered to design the labeling strategy & taxonomy

I wasn't labeling data myself — but guided:

- what labels matter
- how to measure signal vs noise
- confidence thresholds
- fallback logic

- human-in-the-loop corrections

This gave the model a **cleaner, higher-signal dataset**.

3. Led the cloud data architecture + pipeline

This was my core domain:

- ServiceNow/CAI ingestion via SNS/SQS
- S3 raw → Lambda preprocessing → S3 curated
- embeddings via SageMaker
- DynamoDB/Opensearch for fast retrieval
- feature store patterns for future retraining

The ML team could focus on modelling because data architecture was stable.

4. Co-developed model evaluation and MLOps guardrails

Using my DS training, I introduced:

- precision/recall per category
- silhouette scores for clustering quality
- drift detection signals
- low-confidence fallback → human triage

This ensured the model had production quality standards.

5. Drove adoption

I helped translate ML outcomes into business language for Ops/SRE:

"90% of this noise is due to inconsistent free-text + missing categories.

SCOUT groups them into 8 explainable clusters — that's why repeat incidents fall."

R — Result

- ML model delivered faster due to strong E+DS pairing

- First clean, unified incident dataset across SNOW + CAI
 - Clustering revealed **7–9 themes** responsible for most noise
 - ~30–40% misrouting / noise reduction
 - Team adopted cloud-native AIOps foundation for future work
 - I strengthened my DS hands-on credibility and became go-to for ML partnerships in the org
- ▼ Tell me about a time you had to influence stakeholder
- Did by getting data from users, conducting workshop to see what the users really wanted vs what they are asking me to do.
- ▼ Tell me about a time you managed an under performer

• **Answering "Tell me about a time you managed an underperformer":** This narrative offers a perfect structure for demonstrating growth and a systematic approach. "I had a team member who was struggling to meet expectations on a critical project. My initial instinct was to jump in and fix the work myself to protect the timeline. However, I recognized that was a short-term fix that would lead to my own burnout and wouldn't help them grow. Instead, I used a framework that starts with a simple question: 'Would I enthusiastically rehire this person?' The answer was yes, but they clearly needed support. So, I implemented the 'Coach Them Up' strategy. We co-created a 30-day development plan with specific, measurable goals. I paired them with a senior mentor for daily technical guidance and we had twice-weekly check-ins to track progress. Within a month, their performance had turned around completely, and they became a solid contributor to the team."

"Tell me about a leadership mistake you've made": Use the first part of the story to show self-awareness and learning. "A mistake I made early in my management career was reverting to an individual contributor under pressure. An engineer on my team was behind, and I worked all weekend to rewrite their code to hit a deadline. I succeeded in shipping on time, but I failed as a manager. I burned myself out, I failed to help my report grow, and I didn't solve the underlying performance issue. That experience was a powerful lesson. It

taught me to always address the root cause, which led me to adopt a more structured 'Coach Them Up or Coach Them Out' framework to handle performance issues strategically."

▼ Tell me about a time you managed an under performer (PIP)

Situation:

About a year ago, one of my direct reports, *Nate*, was consistently underperforming as a software engineer on a critical delivery team. He frequently missed sprint deadlines, introduced recurring code quality issues, and failed to follow through on action items despite several rounds of informal coaching and feedback. His performance was starting to affect the rest of the team — other engineers were spending time reworking his code, and morale was slipping.

Task:

My responsibility as his manager was to help Nate either succeed or make a fair, transparent decision if improvement didn't happen. I needed to design a structured **Performance Improvement Plan (PIP)** that clearly outlined expectations, support mechanisms, and measurable success criteria, while ensuring full HR compliance and documentation. The goal was to give him every possible opportunity to turn things around within a 90-day period.

Action:

I worked with HR to draft a **SMART-goal-based PIP**. The plan required:

- Reducing code defects from an average of 12 per sprint to fewer than 5,
- Completing all assigned stories by sprint-end for at least 3 consecutive sprints, and
- Participating in pair programming sessions twice weekly with a senior engineer.

I also committed to weekly one-on-one coaching, code review feedback, and bi-weekly progress check-ins with HR.

Throughout the process, I provided written summaries of our meetings, recognized incremental progress, and reinforced the message that the PIP was

designed to help him succeed — not punish him.

Unfortunately, after 60 days, metrics still showed little improvement. Despite the resources provided — including training and mentorship — deadlines were still missed, and QA reported continued quality issues. At the 90-day mark, we met again to review the outcomes objectively. The data and documentation were clear that performance expectations hadn't been met.

Result:

After consulting HR, I made the difficult decision to terminate Nate's employment. The offboarding was handled professionally and respectfully — with a focus on transparency, documentation, and protecting team morale.

The outcome had two lasting effects:

1. **Team performance rebounded quickly**, with sprint velocity improving by nearly 20% within the next cycle.
2. It reinforced a culture of accountability and fairness — the team saw that performance issues would be addressed thoughtfully, with real opportunities for improvement but also with clear consequences.

▼ “Tell me about a time you misjudged someone on your team and what you learned.” (wrong assumption..)

<https://medium.com/@anirudh.banger/a-tale-of-two-engineers-bc93b3e2ba95>

S – Situation

A few years ago, shortly after I became an Engineering Manager, two new engineers joined my team during the final stages of a major release. Both were experienced hires. During standups, one came across as a “rockstar” — confident, proactive, constantly communicating progress. The other was quieter and more cautious, often double-checking his work before sharing updates.

Over time, my instinct told me the first was a high performer and the second might be struggling.

T – Task

As our performance review cycle approached, I needed to accurately assess their contributions. I was at a critical juncture — do I act on my strong

impression and flag the quieter engineer as an underperformer, or do I test my assumption to make sure I'm being completely fair?

A – Action

Before deciding, I pulled objective data — number of defects fixed, average resolution time, and reopen rates.

The results completely contradicted my gut feeling. Both engineers had nearly identical outputs — roughly forty defects each, with only about a one-day difference in turnaround.

It was a humbling realization: I had mistaken a difference in communication style for a difference in performance. The problem wasn't a **performance gap** — it was a **signal-visibility gap**.

Instead of initiating a performance plan, I met with the quieter engineer, shared the data, praised his strong results, and gave coaching on proactively communicating progress so his work would be more visible.

R – Result

Within weeks, he began sharing updates more confidently and became one of our most trusted developers.

More importantly, it permanently shaped how I lead. I learned that confidence and competence aren't the same thing. Since then, I treat instinct as a **hypothesis**, not a **conclusion** — I always validate it with data before making judgments about performance.

That discipline has made me a more objective, fair, and trusted leader.

▼ Onboarding

<https://medium.engineering/engineering-onboarding-processes-at-medium-368095116ac3>

https://www.codepath.org/news/engineering-onboarding-guide?utm_source=chatgpt.com

▼ Encouraged Team Innovation

(Hook)

One thing I've learned across teams is that innovation rarely happens by accident — it needs structure and safety. I've always believed *there's almost always a better way* to do something.

(Inspiration)

Back at **Mastercard**, we had a week-long program called *Take Initiative*, where teams stepped away from their day jobs to brainstorm and prototype ideas that could move the business forward. I saw firsthand how energizing that was — how it unlocked creativity and ownership.

(Transition)

When I joined **Citi**, we were deep into a large modernization program, but I noticed people were doing what was convenient. No one wanted to experiment — everyone was afraid of breaking something. I wanted to bring that same “Take Initiative” spirit here, but in a lighter, more continuous form.

(Action)

So I introduced **biweekly “innovation spikes”** — short, alternating-Friday sessions where engineers could prototype automation ideas, API improvements, or UX enhancements they were curious about.

I made it clear that success was measured by *learning velocity*, not perfection.

We celebrated demos even when experiments failed, and I worked with product leadership to ensure these sessions were treated as **strategic investments**, not distractions.

(Result)

Within a few months, the team's confidence changed completely.

One spike led to a **GraphQL prototype** that simplified complex REST data flows — cutting redundant API calls by over 50% and improving client performance.

Another explored **gRPC** for internal service communication, reducing latency for high-throughput workflows.

What started as a small experiment evolved into a **formal API modernization initiative**, endorsed by our architecture leadership.

(Reflection)

The biggest win wasn't the technology itself — it was the cultural shift. The team moved from “what's the easiest way?” to “what's the best way?”

And for me, it reinforced that fostering innovation isn't about hackathons or slogans — it's about creating a safe, repeatable space where engineers can think differently and turn curiosity into impact.

▼ Conflicting requirements

Situation:

While leading the CAI portal modernization, our product roadmap was prioritized for backend migration and UI polish. Midway through, I analyzed customer usage data and found most complaints weren't about aesthetics. They were about the inefficiency of the search, which returned poor results and required multiple steps to find relevant items.

Task:

I believed we needed to pivot part of the roadmap to **prioritize fine-tuning the search**, which would not only improve relevance but also enable **deep-linking** directly to the right sections. However, the Product Manager wanted to stick with the existing plan to meet quarterly feature targets.

Action:

1. **Gathered Data:** I collected evidence from customer support tickets, NPS feedback, and analytics, showing that search-related frustration was the **#1 cause of churn** among internal users.
2. **Showed, Didn't Tell:** I shared a lightweight proof-of-concept demonstrating how **our proposed search enhancements** could dramatically improve relevance and response times.
3. **Reframed the Goal:** In our roadmap review, I acknowledged the PM's valid delivery concerns. I then reframed the discussion around our shared goal of **customer retention and long-term adoption** (which was suffering) instead of just internal milestones.
4. **Compromised:** We agreed to re-sequence the roadmap—introducing the new search earlier while pushing a minor UI milestone out by one sprint. I

communicated this trade-off to leadership and adjusted the team's sprint goals.

Result:

The impact was immediate. After the new search and deep-linking went live, the **search success rate improved by 40%**. More importantly, user satisfaction scores related to "ease of use" jumped, and leadership recognized it as one of the most impactful roadmap adjustments that quarter.

Reflection:

The experience reinforced that customer data must guide the roadmap. It also showed that disagreements with PMs are opportunities to co-create better outcomes, as long as you bring evidence and empathy to the table.

▼ Failure story

"Application Dependencies" Failure Story

The Situation:

"We needed to map a complex web of application dependencies for risk and compliance. The problem screamed 'graph database,' and my technical instinct said Neo4j was the perfect fit.

I knew there was a learning curve — both for my engineers, who were strong Java developers but new to Cypher, and for our SREs, who'd never supported Neo4j in production. I underestimated just how steep that curve would be.

I believed the power of graph queries would offset the short-term learning tax — and I convinced my team and stakeholders to go with it."

The Failure:

"We spent eight weeks building our first version with Neo4j and a Java API. When we launched it to a pilot group of architects, it was a functional failure.

It was brittle — our Cypher queries worked but were inefficient and hard to debug. Small feature changes took days instead of hours.

It was an operational nightmare — no standard monitoring, no automated backups, and when it ran out of memory, nobody knew how to triage it.

We missed our first major deadline for the attestation dashboard. The project was flagged *red* on our quarterly roadmap. I had stakeholders at my desk asking why a 'simple dependency map' was taking months. Everyone knew we were stuck."

The Fix:

"My failure was over-indexing on the perfect technical solution and underestimating the human and operational cost. I had pushed my team and SRE partners onto an unsupported island, and we were sinking.

I went to my director and stakeholders and said plainly, 'I made the wrong architectural call.' It was a humbling moment, but it reset trust.

We pivoted to a pragmatic solution — a JavaScript-based visualization layer powered by our existing Java and Postgres stack.

Velocity returned almost overnight. Within two sprints, we delivered a version that worked, scaled, and finally gave the compliance team the attestation dashboard they needed."

The Learning:

"It taught me that moving fast is easy — moving fast in the *right direction* requires deep awareness of your team's capacity, not just a technology's potential.

Since then, I never adopt new tech without a capability and operational readiness check. The right solution is the one the organization can *sustain*, not just admire."

▼ Conflict at work

Conflict Story: Kafka vs. Solace — Earning Alignment in a Divided Room

Situation

As part of modernizing Citi's enterprise approval workflow platform — migrating from a legacy **.NET + TIBCO EMS** stack to **Java microservices** — we had to select a new messaging backbone.

Both **Solace** and **Kafka** were enterprise-approved technologies, but for different use cases:

- **Solace** was the sanctioned replacement for TIBCO EMS, ideal for JMS-style request/reply workflows.
- **Kafka** was approved for asynchronous event streaming and already operationally hardened across several modernization programs, including one I previously led.

I proposed Kafka for this migration to avoid introducing another broker, reuse our existing skillsets, and maintain architectural coherence across the enterprise.

However, the **Architecture Review Board (ARB)** pushed back strongly. Their position was:

"This platform handles transactional approval flows — Solace provides built-in JMS semantics and is a like-for-like replacement for TIBCO. Kafka isn't designed for this."

The tension was real: the ARB had valid points about transactional reliability, while I saw the operational and long-term simplification benefits.

Task

My goal wasn't to win a debate — it was to get to a decision that was both **technically sound** and **organizationally safe**.

At that point, about half the ARB supported Kafka, mostly engineers familiar with the stack. The rest — particularly compliance and middleware leads — were unconvinced.

I needed roughly **70% approval** to move forward, and we weren't there yet.

Action (Round 1 — Data and Diplomacy)

I began by validating their perspective and proposing a **side-by-side proof of concept**:

- Same workflow executed on both **Solace** and **Kafka**.
- Measured **latency, ordering, replay integrity, and audit traceability**.

- Shared observability dashboards for full transparency.

The data came back strong for Kafka — slightly faster, cheaper to operate, and meeting all delivery guarantees.

But when I presented the results, several members still resisted.

One compliance architect said, *"It's impressive, but it doesn't feel like an audit-safe system."*

That comment landed with me — I realized the issue was no longer technical; it was about **institutional comfort and trust**.

Action (Round 2 — Re-equip and Re-engage)

I regrouped. Instead of pushing harder, I decided to **expand the lens** and meet them where their concern lived — in *risk assurance*, not architecture diagrams.

1. Shifted from technology to controls language

I mapped every Kafka event flow to **Citi's audit control IDs**, showing exactly how Kafka's idempotent producers and topic compaction satisfied SOX and OCC evidence requirements.

That reframing turned the discussion from "is Kafka risky?" to "how does Kafka meet our existing controls?"

2. Invited a neutral voice into the room

For the next review, I asked one of Citi's **Distinguished Engineers — a Managing Director and Citi Fellow** — to join the session.

He wasn't there to co-present or advocate; I simply wanted his **independent perspective** as someone deeply respected by both camps.

His presence immediately changed the tone — the discussion became less about positions and more about trade-offs and design maturity.

3. Proposed structured guardrails

I recommended a **controlled-use model**:

- Kafka could be used for workflow systems under specific configuration patterns.

- Each use case would undergo a post-implementation compliance review.
- A rollback path to Solace would remain documented for the first six months.

That combination of transparency and safety allowed even hesitant members to support a limited rollout.

Result

The revised proposal passed with **over 75% approval**.

We deployed the first Kafka-based approval flow under the controlled-use model, processing millions of messages monthly with zero data loss and full audit traceability.

While a few ARB members remained cautious, the success of the pilot and the Citi Fellow's endorsement helped institutionalize Kafka as a **strategic alternative for transactional systems**, used safely under governance.

Learning

"That experience taught me that alignment isn't about winning consensus — it's about earning trust.

I learned that sometimes you have to pause, translate your argument into the language others trust, and invite respected perspectives into the room — not to overrule anyone, but to widen confidence.

The real mark of influence is when the outcome feels co-owned, not won."

"Solace is natively audit-safe and transactional — perfect for deterministic approval flows.

Kafka offers more scalability and openness, but reaching the same audit assurance requires added observability, RBAC layering, and control mapping."

⚙️ Solace vs. Kafka — Key Technical Differences

Dimension	Solace	Kafka	Impact / Interpretation
Messaging Model	JMS-style broker with store-and-forward reliability	Distributed log, consumer-managed offsets	Solace = predictable transactional delivery; Kafka = scalable but less deterministic
Delivery Guarantees	Built-in exactly-once via broker transactions	Configurable exactly-once (requires idempotent producers + transactions)	Kafka can match reliability, but needs engineering discipline
Visibility & Control	Broker tracks every consumer acknowledgment	Offsets tracked externally; audit trails must be built	Solace = “audit-visible” out of the box; Kafka = requires instrumentation

📋 Compliance & Audit Considerations

Area	Solace	Kafka	Implication
Traceability	Full message and acknowledgment logs available natively	Must log consumer commits or offsets manually	Kafka needs added observability to reach audit parity
Access Control	Fine-grained RBAC and topic-level ACLs (LDAP/AD integrated)	ACLs are cluster-scoped; enterprise RBAC often layered	Solace offers native compliance alignment
Regulatory Familiarity	Matches existing JMS-based audit models	Requires mapping to control IDs and evidence plans	Solace feels “audit-safe”; Kafka requires documentation and trust-building

Governance

Situation (Add Context)

“As we designed DRIFT — a real-time data fabric to replace our aging batch pipelines — one of the toughest debates came from our governance partner.

Their monitoring and verification processes relied on batch characteristics — they could detect anomalies when daily record counts deviated sharply from

expected baselines. For example, if a domain consistently sent 50K records per day, and that suddenly dropped to 5K, their batch-based validation would immediately flag it for investigation."

◆ Conflict (Highlight the Pushback)

"When we proposed moving that domain to streaming, they pushed back strongly.

'In streaming, there's no end-of-day boundary. How do we know if a drop is a real issue or just data still in flight?'

They feared losing their safety net — and rightly so, because those checks often caught systemic data issues before they reached auditors."

◆ Leadership and Resolution (Show How You Bridged It)

"Instead of dismissing their concern, I reframed it: the goal wasn't *to preserve batch*, it was *to preserve trust*.

We collaborated to design a streaming-native verification model using **sliding windows and temporal aggregates** within Flink.

We introduced hourly checkpoints that computed record counts over rolling windows and compared them against dynamic historical averages stored in Iceberg.

If deviations exceeded 20%, the system emitted real-time alerts through Prometheus — effectively recreating the anomaly detection they depended on, but continuously instead of nightly."

◆ Result (Drive Home the Impact)

"That design convinced governance to approve the transition.

Not only did they retain anomaly detection fidelity, they gained earlier detection — identifying data drift within an hour instead of waiting for the next day's batch.

Technical

Situation

Our core infrastructure platform, responsible for ingesting and analyzing all server, network, and authentication logs, was built on 6-hour batch-based Hive/Spark pipelines.

This latency created a critical **security and compliance blind spot**. If a system was compromised, our Security Operations (SecOps) team wouldn't know for up to 6 hours. This delay meant we were at constant risk of failing our **SOC 2** and **ISO 27001** compliance controls, which mandate timely threat detection and response.

We proposed moving to a real-time fabric (Kafka, Flink, Iceberg) to cut this "time-to-detect" from hours to under 2 minutes. The Architecture Review Board (ARB) blocked the initiative, mandating we stay with Spark Structured Streaming, citing operational overhead and a lack of in-house Flink expertise.

Task

The debate was deadlocked: my team's innovation vs. the ARB's mandate for stability. My role was to break this impasse. I had to reframe the conversation: this wasn't about "new tech vs. old tech." It was about proving that Flink wasn't a *technology* risk, but a controlled, evidence-based solution to a clear and present *business* risk (a potential breach or a failed audit).

Action

I first completely changed my approach. I stopped arguing Flink's technical superiority and instead **acknowledged the ARB's risk-framing as valid**.

I told them, "You are 100% right to be concerned. Let me provide the data you need to feel safe."

I proposed a data-driven partnership instead of a debate. To further de-risk the change, I offered a controlled adoption model: Flink would run in parallel with Spark on the same production feed, with automatic rollback to Spark if any SLA breached. We also documented clear exit criteria and a decommission plan should Flink underperform.

That framing gave the ARB a safety net—we weren't betting the platform, we were conducting a safe, evidence-based experiment.

I then partnered with SRE and Platform Engineering to design the 4-week A/B pilot:

1. **We ran the *exact same* production log-streaming workload** on both Flink and the ARB-preferred Spark Streaming stack.
2. **We instrumented for their concerns:** The pilot was integrated with our standard Prometheus/Grafana stack to transparently measure what *they* cared about: P99 latency, cost (node count), fault-tolerance, and SLA compliance.
3. **We made them validation partners:** I invited the ARB leads to our weekly pilot reviews, turning them from external gatekeepers into internal partners in the validation loop.

Over four weeks, the data provided the "safe" evidence they needed: Flink achieved 99.9% uptime, reduced latency from 6 hours to **under 1 minute**, and required 40% fewer nodes than the Spark job for equivalent throughput.

Result

The data made the decision for us. The ARB unanimously approved Flink for a limited production rollout and, six months later, endorsed it as a new strategic platform technology.

This decision immediately **eliminated our 6-hour security blind spot** and ensured we were fully compliant with our SOC 2 and ISO threat-detection controls.

More broadly, our validation framework was cited by the CTO and adopted by the ARB as the new **blueprint for responsible innovation**, creating a clear, data-driven path for other teams to safely introduce new technologies.

▼ Alignment Over Agreement (Conflict)

Context

I lead a 20-person infrastructure engineering organisation across multiple teams. One of my pods — Pod 4 — was mid-migration, moving our event infrastructure from TIBCO to Solace, which involved rewriting significant portions of the codebase. They'd gone through proper architecture review, agreed on an approach — generating XML payloads directly from code — and were 80% done. A principal engineer from another part of my org needed to

integrate with this work. He reviewed Pod 4's implementation, disagreed with the approach, had a design disagreement with the pod lead that didn't get resolved, and rather than escalating through the right channels, he wrote a parallel implementation inside the same codebase — generating payloads from the database instead. When it came time to merge, the conflict surfaced hard. Two competing implementations, team in-fighting, and a migration deadline at risk. That's when I got pulled in.

Action

I resisted the urge to pick a side or make it a performance conversation straight away. I separated the people problem from the architecture problem because they needed different interventions.

On the architecture side, I brought both parties together and reframed the conversation away from "whose design is better." I pointed out that we now had two implementations doing the same job in the same codebase — and that's not a neutral situation. Every engineer who touches this code carries the cognitive load of understanding both approaches, knowing when to use which, and maintaining that context indefinitely. I also made the case around single source of truth — two competing payload generators might look equivalent today, but they'll silently diverge over time, and nobody will know which one to trust. That reframe shifted the dynamic from a design debate to a sustainability conversation.

On the human side, I protected Pod 4's autonomy deliberately. They had done everything right — architecture review, team consensus, structured execution. If I'd forced them to compromise their design to accommodate a late challenge, I'd have sent a message across my entire org that agreed decisions aren't safe. That kills psychological safety and makes teams stop committing to anything. So the agreed architecture stood.

But I also had a direct conversation with the principal engineer. I told him his instinct to challenge the design wasn't the problem — that's what senior engineers are supposed to do. The failure was in the method. When influence didn't land, he built a workaround instead of escalating. I was direct: the moment you write parallel code to win an argument, you've stopped being an architect and become a liability to the team. We deprecated his implementation and he contributed within Pod 4's standards.

Finally, I put in place a lightweight design review gate for cross-team dependencies so this kind of conflict surfaces at the whiteboard, not at merge time.

Result

The migration completed on schedule. The codebase went forward with a single, agreed implementation. More importantly, Pod 4's confidence in their own decision-making authority was intact — which matters for every future piece of work they own.

Learning

The biggest thing I took from this was the difference between alignment and agreement. You don't need everyone to love the decision — but you need everyone to commit to the path forward. My job wasn't to adjudicate who had the better design. It was to create the conditions where the team could move with clarity and without resentment. The other thing I reinforced for myself is that influence is a skill, not a status. When a senior engineer bypasses it, that's a coaching conversation — not just a process failure.

▼ How do you measure the engineering productivity of your teams

I don't measure lines of code or hours worked. I use the DORA framework to measure delivery performance, layered with business and operational metrics to measure actual value. My goal isn't to make people work harder — it's to find where the system is blocked.

Daily, I'm looking at operational health: the sprint board for blockers, CI/CD pipeline status, active incidents. Why are tickets piling up in code review? Is the build red?

Weekly, I check DORA metrics — specifically lead time from commit to production and deployment frequency. This tells me if our tooling and processes are actually working.

Monthly and quarterly, I zoom out to change failure rate and overall efficiency. Speed means nothing if we're shipping bugs or burning cash.

And across all of it, I track what I call a "say/do ratio" — not to punish the team, but to check planning accuracy. If we consistently plan 50 points and deliver 20, that's a planning problem, and that's where I dive deep.

▼ How do you ensure roadmap is realistic

I start with a **capacity model** based on real velocity — I never plan at 100% because things always come up. Support tickets, on-call, unplanned work — I bake that in from the start.

Before we lock timelines, I **de-risk early**. That means running spikes or walking skeletons on anything uncertain, and mapping all upstream and downstream dependencies so we're not surprised mid-sprint by another team's delay.

And the roadmap is never a fixed document — it's a **living artifact**. I re-forecast every two weeks based on updated velocity, emerging risks, and cross-team dependencies. If something shifts, I'd rather surface it early than deliver a surprise late

▼ The "Architecture Alignment" CARL Story Greg Story

Context

"In my role as a Senior EM at a major financial institution, I inherited a legacy team structure characterized by significant internal friction. A long-tenured peer acted as a technical gatekeeper, frequently obstructing project velocity by citing vague 'architectural misalignments.' This created a bottleneck for my team's deliverables and led to a perception from leadership that the department was struggling with communication and missed deadlines."

Action

"Instead of escalating the friction or bypassing the stakeholder—which would have further damaged the team culture—I took a consultative approach. I initiated a formal '**Architecture Alignment**' series, inviting the peer to lead the definition of our 'ideal state' standards.

I facilitated these sessions by:

- **Translating technical jargon into Business Outcomes:** Shifting the conversation from 'DB access' to 'mitigating delivery risk for Q3 targets.'
- **Implementing a live RACI Matrix:** Moving verbal 'No's' into a transparent, documented framework visible to our Director.
- **Establishing a 5-minute Acknowledgment Rule:** To kill the narrative of unresponsiveness, ensuring all stakeholder pings were acknowledged

immediately, even if the resolution was scheduled for later."

Result

"This strategic 'reset' effectively neutralized the interpersonal conflict by making project blockers visible and data-driven rather than personal. Within six weeks:

- We successfully documented our entire architectural roadmap, which unblocked a critical database integration that had been stalled for months.
- My Director noted a measurable 'reduction in team temperature' and increased her trust in my leadership.
- I reclaimed full ownership of the project intake process, resulting in a **20% increase in team velocity** and a cleaner reporting line to leadership."

Learning

"This experience reinforced that at the Senior Management level, **transparency is the best antidote to politics**. I learned that by treating an obstructionist stakeholder as a consultative partner and centering the conversation on Business Value, you can move the needle on delivery without needing to rely on organizational authority."

Shaped it into diff pods, different cap[ability went into diferent

I got different things I'm delivering across teams, so what I did was I split them functionally, and created pods around different capabilities and functions. And gave ownership within that to support them to be succesful so they are not multi tasking across multiple different priorities

"In my experience, **Culture is what happens when you aren't in the room**. I build runbooks, golden paths, and safe-deployment patterns so that the culture of 'High Standards' and 'Psychological Safety' persists even when I'm on vacation."

2 Performance Management & Feedback

Story 1 – Turning Around an Underperformer

- **Situation:** A mid-level engineer missed sprint goals and had frequent QA rejections.
- **Task:** Improve performance without demoralizing them.
- **Action:** Conducted candid 1:1 to uncover root cause (low confidence + unclear expectations). Co-created a 60-day plan with measurable goals, paired them with a mentor, and did weekly check-ins.
- **Result:** Defect rate dropped 35%, delivery reliability improved, and they later took ownership of a core service redesign.

Story 2 – Giving Hard Feedback to a Senior Engineer

- **Situation:** Senior engineer merged unreviewed PRs to “save time.”
- **Task:** Address behavior without alienating them.
- **Action:** Used SBI feedback: “In the last 2 sprints, 3 merges skipped review, increasing risk.” Discussed impact, listened to bandwidth concerns, and co-designed a fix (review SLA + backup reviewer).
- **Result:** Review backlog cleared, incidents dropped, and trust improved across peers.

3 Career Growth & Mentorship

Story 1 – Guiding a Promotion to Staff Engineer

TCO STORY APPLIES HERE

4 Team Culture, Trust & Psychological Safety

Story 1 – Rebuilding Trust in a Burned-Out Team

- **Situation:** After a failed release, morale was low; engineers avoided speaking up.
- **Task:** Rebuild trust and openness.
- **Action:** Ran retros focused on “what we learned,” not “who failed.” Shared my own mistakes. Invited every member to speak; thanked early risk-callouts. Instituted “no-blame postmortems.”
- **Result:** Within 2 sprints, engagement scores rose; team began flagging risks proactively, cutting late issues by 30%.

Story 2 – New Team Integration

- **Situation:** Took over a legacy platform team with skepticism toward new management.
- **Task:** Build credibility quickly.
- **Action:** Held intro 1:1s to understand frustrations, fixed one persistent blocker in week one, and made workload visible with Kanban.
- **Result:** Delivered two quick wins, earned trust, and team NPS rose from -20 to +45 in one quarter.
-

6 Influencing Without Authority

Story 1 – Driving Platform Standards Adoption

- **Situation:** Multiple teams resisted adopting a shared CI/CD standard.
- **Task:** Influence adoption without mandate.
- **Action:** Collected data on build failures, ran a pilot with one team, published success metrics (build time ↓40%). Shared results in open forums.
- **Result:** 4 more teams adopted voluntarily within a quarter; leadership later made it org standard.

Story 2 – Aligning Product and Engineering Priorities

- **Situation:** Product wanted rapid feature rollout; engineering needed time for reliability.
 - **Task:** Align priorities without escalation.
 - **Action:** Mapped customer pain points, quantified incident impact, proposed a balanced roadmap. Presented shared gains: "Fewer rollbacks = faster net delivery."
 - **Result:** Joint plan approved, incidents halved, and feature throughput increased 20% sustainably.
-

Hello Interview Behavioral list

Conflict resolution WILL COME OUT.

- Most times disagreement is a path to better outcomes
- Embrace not avoid conflict
- They want to access if you will make the team and people better.
- Can you navigate a problem with empathy?
- Can you solve a problem looking for a win win solution.
- can you focus on business outcome while having these conflicts.
- Also checking cross communication skills.
- They are also checking for levels you are operating at. (what level are you operating at, at your company).

What story to choose. When choosing a story of conflict from your past? (SM level - Organisational in nature)

- High stakes conflict should be picked. (high Outcome)
- Involve you deeply. You must be part of the argument deeply.
- Choose one where you were right.

aGiven a response. - Repeatable actions that you did, which you will bring to their company.

STAR or CAR - Context ACTION(60%) and Result. (result should show you got what you wanted but not in a damaging way, whereby the person no longer wants to work with you again also , result should come with some learnings.)

In given a great response like that, I want to see that you understood what the other person was looking for. and show signs of empathy from the other person's side as well. Not just interested in running through wall.

Architectural choice should come with some PoC to show

Prioritize should come with Data to push the discussion.

- in all of these, there are communications, they want to see how you do that communication too, and how you push the channel and leverage and move the conversation forward is crucial too
- try to get some win win, might not be easy, but try to get the conflict to preserve the relationship. dont vilify but rational.
- dont pick story where the other side was doing something obviously stupid, it leans towards you too much and makes it imbalance.

Inter department conflict.

- Systemic solutions that ensure there is a way path for similar conflict next time.
- You dont bring senior folks in too early and not when things go down too late.
- Sometimes a backstory could help. But be concise in your communication. Efficient context is important too.

NEW QUESTION(Conflict) → tech companies want to see you can disagree respectfully. disagree and commit.

Disagree with your manager. (Process that needs to chain)

Hard feedback - I gave to Ankur about his pod. Too much yes to what product is asking them vs keep eye focussed on outcome. He thinks coming to meetings and saying, how that was the requirement we got was going to be good enough. No its

not. You are an active participant and a stakeholder in this. When this requirements comes for your pod, I expect you to have a sit down

Dont bring up things like

- Griveances
- didnt get promotion
- avoid trivial issues
- avoid conflict where you just dont like the idea, nothing tangible.
- Avoid emotional stories with not much logic.

DRIFT is the Story - conflict and planning is from DRIFT.

Set the stage early and on very strong foot.

Tell me about yourself and most impactful project.

Good questions to ask back, you must ask.

product direction team is tking

tech stack

Tell me about yourself- framing - (past, present, future)

- Who you are, my name is and i have been
- Enjoy mentoring people, type grow
- What you are working on right now.(hot stuff)
- End is - what you want to get from the role. tie the ending to what the team is looking for

Why did you become a manager: -

I enjoy technical work but I am also highly interested the output/product of what we are building on top of that, I get

a lot of fulfillment and energy out of helping people accomplish more together than they might on their own. I initially started as what you will call Tech Lead manager at mastercard, then scope kept exapandingout from there.

Questions That I am looking for Solid answers for

- How do you deal with conflict? - Give an example of conflict you dealt with - something big scale, architectural and impactful - how did you resolve it- give example - 100%
- Situation where you didnt agree with your supervisor - 50%
- Project you are most proud of - 100%
- Dealt with underperforming engineer - 80%
- Tell me about a project which you failed - -70%
- Give an example of a calculated Risk you took. -
- hardest Feedback you've ever had to give.
- A situation where you worked cross functionally to remove a significant barrier for your team, or situation in which you were able to really maximize productivity of a team by digging into details
- Planning in projects. how do you break things down.

More questions that I Need to deal with

(eBay)

- So maybe tell me about a time when you had to approach breaking down complex project features. into more manageable subtasks, and maybe just explain to me what method do you use to do this?

- Maybe tell me about a project where your work had a measurable impact on the business and how did you evaluate its success?
- Maybe give me an example of when you either display these values or champion these values within your team or fostered innovation within your team, .
- Have you facilitated any strong onboarding experience? Maybe give me some examples about how you hire and then how you kind of onboard people and integrate them into a team
- Tell me about a time when you had to motivate a struggling team member. How did you approach the situation, what was the result?
- Maybe describe a situation where you had to assess multiple competing priorities. How did you determine the best course of action, how did you communicate with the relevant teams?
- Tell me about a hard technical problem you worked on
- Tell me about your process for hiring engineers. What specific qualities do you look for?"
- "Describe a time you hired someone who was a fantastic addition to the team. What made them great, and how did you identify that during the interview process?"
- "How do you approach building a diverse and inclusive team?"
- "Tell me about a hiring mistake you made. What did you learn from that experience?"
- How do you build trust with a new team or with new members joining your team?"
- "Describe a time your team's morale was low. What did you do to diagnose the cause and address it?"
- "Tell me about the most complex system you have been responsible for. What were the biggest challenges in its design or ongoing operation?"
- teamwork

- Influencing
- Ethical/moral conflict

Pillar 1: People Management and Leadership

Hiring and Talent Acquisition

- Tell me about your process for hiring engineers. What specific qualities do you look for?
 - Describe a time you hired someone who was a fantastic addition to the team. What made them great, and how did you identify that during the interview process?
 - How do you approach building a diverse and inclusive team?
 - Tell me about a hiring mistake you made. What did you learn from that experience?
-

Performance Management

- Tell me about a time you had to manage an underperforming engineer. What was the situation, what steps did you take, and what was the ultimate outcome?
 - How do you ensure your high-performing engineers remain challenged, engaged, and motivated?
 - Describe your approach to giving difficult feedback. Can you provide a specific example of when you did this?
 - Have you ever had to terminate an employee? Walk me through that process and your role in it.
-

Career Growth and Mentorship

- How do you support the career growth and professional development of your team members?
 - Tell me about a time you successfully helped an engineer achieve a promotion or a significant career goal. What was your specific role in their success?
 - How does your approach to mentoring a junior engineer differ from how you would mentor a senior or staff engineer?
-

Team Culture and Motivation

- How do you build trust with a new team or with new members joining your team?
 - Describe a time your team's morale was low. What did you do to diagnose the cause and address it?
 - How do you foster a culture of innovation and continuous improvement within your team?
 - What does psychological safety mean to you, and what specific, tangible actions do you take to create it on your team?
-

- How do you develop and grow engineers on your team
- Tell me about a time you managed underperforming team members
- How do you create a culture of high performance and psychological safety
- Tell me about a time you had to influence without authority
 - Navigating ambiguity and resistance
 - How you aligned stakeholders using data, empathy, or customer impact
 - Avoiding escalation unless necessary
- How do you balance delivery pressure with people wellbeing

- recognition of burn out risk signals
- Pragmatic prioritization (e.g adjusting scope, defending capacity)
- Showing care and accountability - protect the team and meet business goals
- Tell me about a decision you made that significantly improved customer experience
 - Whether you truly understand customer problems (internal or external)
 - How you gather feedback and convert it into technical or process improvements
 - Tangible Impact - Improved satisfaction, engagement, or reduced friction
- How do you ensure your teams work aligns with business and customer outcomes?
- Tell me a time you had to manage an underperforming engineer
 - Ability to give direct, kind feedback, whether you follow a structured process (e.g PIP)
- Describe how you support the career growth of your direct reports. Can you give a specific example.

Pillar 2: Project Execution and Strategic Delivery

Prioritization and Roadmapping

- How do you balance the need to deliver new features against the necessity of addressing technical debt?
 - Tell me about a time you had to make a critical decision on project priorities with incomplete data. How did you approach it and justify your decision?
 - Walk me through your process for roadmap planning for a quarter or a six-month period.
 - Describe a situation where you had to say “no” to a compelling request from a product manager or a key stakeholder. How did you handle that conversation?
-

Managing Ambiguity and Risk

- Tell me about a project that had unclear or constantly changing requirements. How did you provide clarity and stable direction for your team?
 - Describe the most significant project risk you've had to manage. What steps did you take to identify and mitigate it?
 - Walk me through a time a project was significantly behind schedule. What actions did you take to get it back on track?
-

Delivering Results and Ensuring Quality

- What metrics do you use to measure the success and health of your team and your projects?
 - Tell me about a time you had to make a difficult trade-off between speed of delivery and the quality of the final product. How did you decide, and what was the outcome?
 - How do you ensure your team maintains a consistently high standard of quality in their work?
 - Describe your experience with project budget management and resource allocation.
-

Pillar 3: Technical Acumen and Architectural Vision

Guiding Technical Decisions

- Tell me about a time your team had a significant disagreement on a technical approach. How did you facilitate a resolution?
 - How do you balance your hands-on technical expertise with your managerial responsibilities? How deep in the code do you typically stay?
 - Walk me through a complex technical decision your team made recently. What were the trade-offs you considered?
-

System Design and Scalability

- Tell me about the most complex system you have been responsible for. What were the biggest challenges in its design or ongoing operation?
 - Describe a time you had to lead your team through the process of scaling a system to handle a significant increase in traffic or data volume.
 - How do you approach long-term architectural planning and vision for your team's services?
-

Innovation and Technology Adoption

- How do you stay up-to-date with the latest technologies, frameworks, and industry trends?
 - Tell me about a time you successfully introduced a new technology or a significant process change to your team. What was your approach to gain buy-in and ensure a smooth adoption?
 - How do you decide when it's appropriate to build a solution in-house versus buying a third-party tool or using an open-source solution?
-

Pillar 4: Cross-Functional Influence and Conflict Resolution

Stakeholder Management

- Tell me about a time you had to communicate a complex technical concept or a project update to a non-technical audience. How did you ensure they understood?

- Describe a time you had a significant disagreement with a product manager on priorities or features. How did you work through it to find a resolution?
 - How do you manage expectations from stakeholders when project timelines shift or the scope needs to change?
-

Conflict Resolution

- Tell me about a conflict that arose between two members of your team. How did you handle it?
 - Describe a time you disagreed with a decision made by your manager. What did you do? What was the outcome?
 - Tell me about a time you had to mediate a conflict between your team and another engineering team.
-

Collaboration and Alignment

- How do you collaborate with your product and design counterparts to ensure the team is building the right product in the right way?
 - Tell me about a successful cross-functional project you led. What specific actions did you take that made it successful?
 - How do you handle receiving a top-down directive that you or your team do not agree with?
-

Part III: The Senior Engineering Manager – Scale, Strategy, and Influence

Managing Managers and Organizational Design

- How do you measure the success and effectiveness of the engineering managers who report to you?
- Tell me about your approach to mentoring and developing other managers. What areas do you focus on?
- Describe a time you had to structure or reorganize multiple teams to achieve a new business objective. What was your thought process and what trade-offs

did you consider?

- How do you handle a conflict or a lack of alignment between two of the managers who report to you?
-

Long-Term Strategic Planning and Vision

- How do you ensure your organization's technical strategy is tightly aligned with the company's long-term business goals?
 - Walk me through how you would develop a one- to two-year vision for a new engineering organization you were tasked to lead.
 - Tell me about a time you had to “think big” and pursue a bold direction that had a significant, long-term impact.
-

Scaling Teams and Culture

- Imagine our user base is projected to triple in the next year. How would you prepare your organization's architecture and teams for that scale?
 - What is your philosophy on building and scaling engineering organizations? What are the most common failure modes?
 - How do you ensure a consistent culture and set of engineering practices across multiple teams, especially in a distributed or remote environment?
-

Budget, Headcount, and Cross-Org Influence

- Describe your experience with budget management and headcount planning for an organization of [X] size.
- Tell me about a time you had to advocate for significant resources—budget, people, or time—for your organization. How did you build the business case to secure them?
- Describe a time you led a major cross-organizational initiative. How did you drive alignment and manage dependencies with teams and leaders you didn't directly control?

▼ If you were the first Data Engineering Manager here, how would you design the initial data platform in your first 90 days?

- First 30 Days (The Foundation): → Initial focus must be on establishing a MVP. This involves making a pragmatic choices for core components
 - Core Components (Pragmatic choices)
 - Cloud data warehouse: Google BigQuery or Snowflake
 - Data ingestion: Managed tool like Fivetran or simple framework for batch APIs
 - Transformation layer: dbt
 - Orchestration: Lightweight tool like Airflow or Prefect
 - Approach highlights
 - Aligns with modern ELT (Extract, Load, Transform) paradigm
 - Bias for action: prioritize speed to value
 - Managed services preferred to accelerate delivery
- Next 30 days (Delivering Value):
 - Focus shift: From plumbing → to business impact
 - Key actions:
 - Collaborate with stakeholders → define tracking plan for key events
 - Ship core metrics mart → answer critical business questions (revenue, activation, retention).
 - Outcome demonstrated: Ability to connect infrastructure investments → to tangible BI & analytics outcomes.
- Final 30 days (Scaling and Governance) - > Long-Term Success
 - Stability & trust: Establish SLAs for data freshness and quality
 - Security: Implement role-based access control
 - Accessibility : Roll out self-service BI tool with semantic layer for non technical users

- Documentation: Centralize in a data catalog
- Demonstrated Capability: Show foresight → long-term maintainability, security, scalability and data democratization

▼ Build vs Buy

1. Default to buy for Commodity Functions

- For **undifferentiated, standard components**, buying is usually the right move.
- Examples:
 - **Data ingestion** from SaaS sources → Fivetran.
 - **Orchestration** → Airflow or managed equivalent.
 - **Observability** → Datadog or similar.
- Rationale: **Developer hours are expensive**; they should be invested in **business-specific problems**, not reinventing commodity solutions.

2. Build for Core competitive advantage

- **Build only when it contributes to IP or lasting competitive edge.**
- Example:
 - High-frequency trading firm → builds **ultra-low-latency messaging system**.
 - Same firm → buys **HR software** (non-core).
- A good manager must separate what is **core vs. context** for their business.

3. Analyze total cost of ownership

- Avoid the trap of **underestimating long-term costs** of custom-built tools.
- TCO includes:

- Ongoing **maintenance & security patches**.
- **On-call support** & operational burden.
- **Feature development & enhancements**.
- Lesson: A SaaS tool that looks pricey upfront may be **cheaper overall** once hidden costs are factored in.

4. Evaluate Vendor Risks

- Risks with buying include:
 - **Vendor lock-in** → difficult/expensive migrations.
 - **Unpredictable price hikes** as usage scales.
 - **Roadmap misalignment** with business needs.
- Strong managers think ahead with an **exit strategy** even when opting to buy.

5. Consider Hybrid Strategies

Advanced candidates may propose **mixed approaches**:

- **"Buy while you build"** → vendor solution for immediate value while building a long-term tailored one.
- **Custom wrapper on open-source tools** → adds flexibility without starting from scratch.

▼ How to manage technical debt

Strong Answer Framework:

1. **Acknowledge, Catalog, and Quantify**

- Make debt **visible** without blame.
- Tie it to **business impact**, e.g.:

Instead of "schema is bad," → "Schema slows queries by 30% and costs 5 dev hours/week."

2. **Prioritize with a Framework**

- Not all debt is equal. Classify it:

- **Debt with Interest Payments** → ongoing pain (e.g., manual deployments) → **high priority**.
- **Dormant Debt** → in stable code rarely touched → can be **deferred**.
- **Risk-Based Debt** → increases likelihood of incidents (e.g., no tests on critical pipeline) → **high priority**.

3. Communicate and Advocate

- Make debt **visible to stakeholders**.
- Frame in terms of **trade-offs** and business costs.
- Run **regular infra health reviews** with product teams to align on long-term vs short-term choices.

4. Allocate Dedicated Capacity

- Tech debt mgmt = **continuous**, not one-off.
- Dedicate **15–20% of sprint capacity** to debt paydown.
- Prevents debt from snowballing into a massive rewrite.

▼ How do you ensure you hire the best people

Clear Definition of “Best”: I align with stakeholders on the critical competencies—technical depth, problem-solving skills, communication, cultural fit, and growth mindset.

Structured & Objective Process: I use a blend of technical evaluations (coding challenges, architecture design scenarios) and behavioral interviews that reveal how candidates approach complexity, collaboration, and adaptability.

Panel Feedback & Calibration: We use multi-interviewer panels, each focusing on different dimensions (technical deep dive, soft skills, leadership potential). Afterward, we calibrate our feedback to minimize bias and ensure we’re making data-driven decisions.

Sell the Vision: High-caliber candidates are often courted by multiple companies. I invest time in sharing our mission (e.g., building the next-gen identity platform or designing secure blockchain solutions). This passion resonates with top talent.

▼ Key ways to improve stakeholder management and communications

- **Stakeholder engagement is a discipline** – The report treats communication as an engineered process. Poorly defined roles and imprecise instructions can cause project delays and failures, echoing data that 56 % of project failures stem from ineffective communication
- **Map and prioritise stakeholders** – Use a structured identification process (e.g., stakeholder register) and apply a power/interest grid to categorise stakeholders. “High Power/High Interest” stakeholders must be managed closely; “Low Power/Low Interest” stakeholders are simply monitored. This dynamic map should be revisited whenever organisational changes occur.
- **Clarify roles with a RACI matrix** – Ambiguity kills velocity. Assigning Responsible, Accountable, Consulted and Informed roles for every deliverable prevents duplicate effort and unresolved tasks. Implementing RACI has been shown to reduce cycle time by up to 15 % and increase throughput moldstud.com.
- **Create a formal communication plan** – Define objectives, audiences, key messages, channels, frequency and ownership for each stakeholder group. Blend synchronous channels (stand-ups, demos) with asynchronous ones (Confluence, Jira dashboards) to respect focus time and provide persistent context. Tools like Jira and Confluence can become a “single source of truth” for progress and decisions
- **Leverage planning frameworks** – Use RICE, MoSCoW, Kano or impact-effort matrices to prioritise features objectively. The EM supplies realistic effort estimates through structured decomposition and facilitates collaborative trade-offs.
- **Build a strong EM-PM partnership** – Regular 1:1s, clear domains of decision-making and a shared ownership of the roadmap are essential. Agree who has final say on scope versus architecture to avoid stalemates.
- **Persuasion through business impact** – Translate technical proposals into business language, use analogies and visualisations, and present a menu of options with trade-offs. Frame technical debt as a “high-interest loan” hampering velocity, and emphasise the cost of delay or risk to revenue
- **Accelerate feedback loops** – Optimise agile ceremonies, turn sprint demos into narrative-driven events, and adopt Meta’s “2-2-2 method” (≤2 hours per

code review, ≤2 feedback rounds, then a synchronous discussion) to cut review delays. Invest in TDD, CI/CD and observability so the system itself provides fast, objective feedback.

- **Iterate and measure** – Roll out changes in phases: assess, design, implement, iterate. Track stakeholder satisfaction, unplanned clarification meetings, time-to-feedback on documents and engineering metrics like cycle time and deployment frequency to prove impact.

Quadrants of the Power/Interest Grid

1. High Power / High Interest → Manage Closely

- Key decision-makers, sponsors, regulators.
- Need active engagement, regular updates, and involvement in key decisions.

2. High Power / Low Interest → Keep Satisfied

- Senior executives who may not be deeply involved day-to-day.
- Don't overload with details, but keep them aligned and satisfied.

3. Low Power / High Interest → Keep Informed

- End users, team members, analysts.
- Regular communication and updates so they feel engaged.

4. Low Power / Low Interest → Monitor (Minimal Effort)

- Peripheral stakeholders.
- Light-touch monitoring; communicate only if relevant.

▼ Something 4

Grouping

Hardest thing I did recently Deep Dive

Deep Dives

Reinforcement Learnings

Self Service Data Onboarding

SCOUT AI

SCOUT Deep Dive